

Convolutional Neural Network থেকে Deep Residual Learning (ResNet) পর্যন্ত

মূল গবেষণাপত্রের ভিত্তিতে প্রস্তুতকৃত

“Deep Residual Learning for Image Recognition”

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun
Microsoft Research, 2015

অংশ ১: CNN-এর ভিত্তি (CNN Foundation)

ResNet (Residual Network) বোঝার আগে Convolutional Neural Network (CNN)-এর মৌলিক ধারণাগুলো ভালোভাবে বোঝা প্রয়োজন। এই অংশে ছবি কীভাবে সংখ্যায় রূপান্তরিত হয়, Convolution কীভাবে কাজ করে, এবং একটি CNN-এর ভেতরে তথ্য কীভাবে প্রবাহিত হয় তা ব্যাখ্যা করা হলো।

১.১ ছবি কীভাবে সংখ্যায় প্রকাশ পায় (Image Representation)

একটি ডিজিটাল ছবি কম্পিউটারের কাছে আসলে একটি সংখ্যার গ্রিড (matrix) মাত্র। যেকোনো ছবিকে তিনটি মাত্রা (dimension) দিয়ে বর্ণনা করা হয়:

Height × Width × Channels

- Height (উচ্চতা): ছবির উল্লম্ব দিকে কতগুলো pixel আছে।
- Width (প্রস্থ): ছবির অনুভূমিক দিকে কতগুলো pixel আছে।
- Channel: প্রতিটি pixel-এ রঙের কতগুলো উপাদান (component) সংরক্ষিত আছে।

উদাহরণস্বরূপ —

ছবির ধরন	Dimension	ব্যাখ্যা
Grayscale (সাদা-কালো) ছবি	$28 \times 28 \times 1$	শুধু একটি channel — প্রতিটি pixel-এ কেবল উজ্জ্বলতা (intensity) মান থাকে
RGB (রঙিন) ছবি	$224 \times 224 \times 3$	তিনটি channel — Red, Green, Blue; প্রতিটি pixel-এ তিনটি সংখ্যা থাকে

এখন লক্ষ্য করা যাক CNN-এর মাঝামাঝি স্তরে দেখা যাওয়া একটি সাধারণ dimension:

$56 \times 56 \times 256$

- ৫৬ = feature map-এর উচ্চতা (height)
- ৫৬ = feature map-এর প্রস্থ (width)
- ২৫৬ = feature map channel-এর সংখ্যা, অর্থাৎ নেটওয়ার্ক এই স্তরে ২৫৬ ধরনের ভিন্ন ভিন্ন প্যাটার্ন (pattern) শনাক্ত করছে

এই ২৫৬টি channel বিভিন্ন বিমূর্ত (abstract) বৈশিষ্ট্য ধারণ করতে পারে — যেমন কোনো একটি channel অনুভূমিক প্রান্ত (edge) শনাক্ত করতে পারে, আরেকটি বৃত্তাকার আকৃতি, আরেকটি টেক্সচার বা রঙের বৈপরীত্য। নেটওয়ার্ক গভীর হলে এই channel-গুলো ধীরে ধীরে আরও জটিল ও বিমূর্ত বৈশিষ্ট্য (যেমন 'চোখ', 'চাকা') শনাক্ত করতে শেখে।

১.২ কনভোলিউশন (Convolution)

Convolution হলো CNN-এর মূল গাণিতিক অপারেশন, যার মাধ্যমে ছবি থেকে বৈশিষ্ট্য (feature) বের করা হয়। এর সাথে যুক্ত পরিভাষাগুলো নিচে দেওয়া হলো:

- Kernel / Filter: একটি ছোট সংখ্যার গ্রিড (যেমন 3×3), যা ছবির উপর দিয়ে সরে সরে (slide) গিয়ে বৈশিষ্ট্য খোঁজে।
- Kernel size: kernel-এর মাপ, যেমন 3×3 বা 5×5 ।
- Stride: প্রতিবার kernel কতটুকু সরে (কয়টি pixel বাদ দিয়ে) যাবে।
- Padding: ছবির চারপাশে অতিরিক্ত (সাধারণত শূন্য মানের) pixel যোগ করা, যাতে output-এর আকার নিয়ন্ত্রণ করা যায়।
- Feature map: convolution-এর ফলাফল হিসেবে তৈরি হওয়া নতুন গ্রিড।
- Input channel: input-এ কতগুলো channel আছে (যেমন RGB ছবির জন্য ৩টি)।
- Output channel: কতগুলো ভিন্ন filter ব্যবহার করা হয়েছে, তত সংখ্যক output channel তৈরি হয়।

মৌলিক প্রবাহ (flow) নিচে দেখানো হলো:

Input Image → Convolution → Feature Map

একটি সংখ্যাগত উদাহরণ দেখা যাক। ধরা যাক input-এর একটি 3×3 অংশ নিচের মতো:

1	2	0
0	1	3
2	1	0

এবং 3×3 kernel:

1	0	1
0	1	0
1	0	1

Convolution-এর নিয়ম হলো — উভয় গ্রিডের একই অবস্থানের মান পরস্পর গুণ করে, সব গুণফল যোগ করা:

$$\begin{aligned} \text{Output} = & (1 \times 1) + (2 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (3 \times 0) + (2 \times 1) + (1 \times 0) + (0 \times 1) = \\ & 1 + 0 + 0 + 0 + 1 + 0 + 2 + 0 + 0 = 4 \end{aligned}$$

3×3 kernel একবার স্লাইড করে একটি output মান তৈরি করে

kernel পুরো ছবির উপর দিয়ে stride অনুযায়ী সরতে সরতে সম্পূর্ণ feature map তৈরি করে।

১.৩ Convolution Output Size নির্ণয়

Convolution-এর পর output feature map-এর আকার নিচের সূত্র দিয়ে নির্ণয় করা হয়:

$$\text{Output size} = \lfloor (N + 2P - K) / S \rfloor + 1$$

N, P, K, S -এর মান জেনে output-এর height/width বের করা যায়

- N = input-এর আকার (height বা width)
- P = padding (চারপাশে যোগ করা অতিরিক্ত pixel-এর সংখ্যা)
- K = kernel-এর আকার
- S = stride

উদাহরণ: ধরা যাক input আকার $N = 7$, kernel $K = 3$, stride $S = 1$, padding $P = 0$ । তাহলে —

$$\text{Output size} = \lfloor (7 + 0 - 3) / 1 \rfloor + 1 = \lfloor 4 \rfloor + 1 = 5$$

অর্থাৎ 7×7 input থেকে 5×5 আকারের feature map তৈরি হবে।

১.৪ একাধিক Channel-এ Convolution

একটি RGB ছবিতে ৩টি input channel (Red, Green, Blue) থাকে। একটি convolution filter একই সাথে তিনটি channel জুড়ে কাজ করে — অর্থাৎ প্রতিটি filter আসলে $3 \times 3 \times 3$ (kernel height $\times$ kernel width $\times$ input channel) আকারের হয়, এবং তিনটি channel-এর গুণফল যোগ করে একটিমাত্র output মান তৈরি হয়।

উদাহরণ:

Input	Conv Layer	Output
$224 \times 224 \times 3$	64টি filter	$224 \times 224 \times 64$

এখানে ৬৪টি ভিন্ন filter ব্যবহার করা হয়েছে বলেই output-এ ৬৪টি channel তৈরি হয়েছে — প্রতিটি filter সম্পূর্ণ input (তিন channel সহ) স্ক্যান করে একটি করে নতুন feature map (একটি channel) তৈরি করে। তাই output channel-এর সংখ্যা সবসময় filter-এর সংখ্যার সমান।

১.৫ Activation Function: ReLU

Convolution-এর পর সাধারণত একটি non-linear activation function প্রয়োগ করা হয়, যাতে নেটওয়ার্ক শুধু সরলরৈখিক (linear) নয়, বরং জটিল ও অ-রৈখিক সম্পর্কও শিখতে পারে। ResNet-সহ অধিকাংশ আধুনিক CNN-এ ব্যবহৃত হয় ReLU (Rectified Linear Unit):

$$\sigma(x) = \max(0, x)$$

অর্থাৎ — ঋণাত্মক (negative) মানকে শূন্য করে দেওয়া হয়, ধনাত্মক (positive) মান অপরিবর্তিত থাকে।

উদাহরণ:

x	$\sigma(x) = \max(0, x)$
-3	0
0	0
2	2
7	7

Activation function ছাড়া, একাধিক convolution/linear স্তর স্তুপ করলেও তা গাণিতিকভাবে একটিমাত্র linear স্তরের সমতুল্য হয়ে যেত — ফলে নেটওয়ার্ক জটিল প্যাটার্ন শিখতে পারত না। ReLU নেটওয়ার্ককে অ-রৈখিকতা (non-linearity) দেয়, এবং এটি গণনাগতভাবে সরল ও দ্রুত হওয়ায় গভীর নেটওয়ার্কে ব্যাপকভাবে ব্যবহৃত হয়।

১.৬ Pooling

Pooling হলো feature map-এর স্থানিক আকার (spatial size) ছোট করার একটি পদ্ধতি, যাকে বলে spatial downsampling। এর ফলে গণনার পরিমাণ কমে এবং নেটওয়ার্ক কিছুটা অবস্থান-নিরপেক্ষ (translation-invariant) হয়।

- Max Pooling: pooling window-এর মধ্যে সবচেয়ে বড় মানটি রাখা হয়।
- Average Pooling: pooling window-এর সব মানের গড় (average) রাখা হয়।
- Pooling kernel ও stride: convolution-এর মতোই window-এর আকার ও তা কতটুকু সরবে তা নির্ধারণ করে।

উদাহরণ — একটি 2×2 Max Pooling (stride 2), input 4×4 :

1	3	2	4
5	6	1	2
0	9	7	8
3	1	2	5

Input-কে 2×2 আকারের চারটি অ-ওভারল্যাপিং (non-overlapping) অংশে ভাগ করে প্রতিটি অংশের সর্বোচ্চ মান নেওয়া হলে output হয়:

6	4
9	8

এভাবে stride 2-সহ pooling ব্যবহার করলে 56×56 আকারের feature map সহজেই 28×28 -এ নেমে আসে — height ও width উভয়ই অর্ধেক হয়ে যায়, কিন্তু channel সংখ্যা অপরিবর্তিত থাকে।

১.৭ Batch Normalization (BN)

Batch Normalization হলো একটি কৌশল, যা প্রতিটি mini-batch-এর জন্য একটি স্তরের output-কে normalize করে — অর্থাৎ গড় (mean) প্রায় ০ এবং ভেদাঙ্ক (variance) প্রায় ১-এর কাছাকাছি রাখে। এর মূল ধারণা:

$$\hat{x} = (x - \mu) / \sqrt{(\sigma^2 + \epsilon)}$$

$$y = \gamma \hat{x} + \beta$$

- μ ও σ^2 হলো ঐ mini-batch-এর গড় ও ভেদাঙ্ক

- γ ও β হলো শেখার যোগ্য (learnable) প্যারামিটার, যা normalization-এর পরে স্কেল ও শিফট সমন্বয় করে
- ϵ একটি ছোট সংখ্যা, যা শূন্য দিয়ে ভাগ এড়াতে যোগ করা হয়

Batch Normalization কেন ব্যবহার করা হয়:

- প্রতিটি স্তরের input-এর বিন্যাস (distribution) প্রশিক্ষণের সময় বারবার পরিবর্তিত হওয়া (internal covariate shift) কমায়
- উচ্চতর learning rate ব্যবহার করা সম্ভব করে, ফলে প্রশিক্ষণ দ্রুত হয়
- নেটওয়ার্ককে আরও স্থিতিশীলভাবে (stable) প্রশিক্ষণ দেওয়া সম্ভব করে

✉ মূল ResNet গবেষণাপত্রে প্রতিটি convolution-এর পরপরই এবং activation-এর আগে Batch Normalization ব্যবহার করা হয়েছে। লেখকরা দেখিয়েছেন, BN ব্যবহার করার পরও অত্যন্ত গভীর plain network-এ degradation problem থেকে যায়— অর্থাৎ শুধু BN দিয়ে গভীরতার সমস্যা সমাধান হয় না। এটিই paper-এর একটি গুরুত্বপূর্ণ পর্যবেক্ষণ, যা পরবর্তী অংশে আলোচনা করা হয়েছে।

অংশ ২: গভীরতা (Depth) কেন গুরুত্বপূর্ণ

CNN-এর একটি মূল বৈশিষ্ট্য হলো এটি স্তরে স্তরে ক্রমবর্ধমান জটিল বৈশিষ্ট্য শেখে। এই ক্রমটি নিচের প্রবাহে দেখানো যায়:

Pixels → Edges → Textures → Shapes → Object Parts → Objects

- প্রাথমিক স্তরগুলো (early layers) কেবল সাধারণ প্রান্ত (edge) ও রঙের বৈপরীত্য শনাক্ত করে
- মাঝামাঝি স্তরগুলো (middle layers) টেক্সচার ও সরল আকৃতি (shape) চেনে
- গভীর স্তরগুলো (deep layers) বস্তুর অংশ (object part) ও সম্পূর্ণ বস্তু (object) চিনতে সক্ষম হয়

যেহেতু প্রতিটি অতিরিক্ত স্তর নেটওয়ার্ককে আরও বিমূর্ত (abstract) ও জটিল প্রতিনিধিত্ব (representation) শেখার সুযোগ দেয়, স্বাভাবিকভাবেই ধারণা করা হতো — নেটওয়ার্ক যত গভীর হবে, তত ভালো ফলাফল পাওয়া যাবে। এখান থেকেই ResNet গবেষণাপত্রের কেন্দ্রীয় প্রশ্নটি উঠে আসে:

☞ “Is learning better networks as easy as stacking more layers?” — অর্থাৎ, শুধু আরও বেশি স্তর জুড়ে দিলেই কি একটি নেটওয়ার্ক আরও ভালো হয়ে যাবে?

বাস্তবে দেখা গেছে, উত্তরটি হলো — না। কেবল স্তর যোগ করলেই কর্মক্ষমতা (performance) বৃদ্ধি পায় না; বরং একটি নির্দিষ্ট গভীরতার পর কর্মক্ষমতা খারাপ হতে শুরু করে। এর কারণ পরবর্তী অংশে বিস্তারিতভাবে আলোচনা করা হয়েছে।

অংশ ৩: গভীর নেটওয়ার্কের সমস্যাসমূহ

৩.১ Vanishing Gradient

প্রশিক্ষণের সময় Backpropagation পদ্ধতিতে gradient (ক্রটির প্রতি সংবেদনশীলতা) নেটওয়ার্কের শেষ স্তর থেকে প্রথম স্তরের দিকে ফিরে আসে। নেটওয়ার্ক অনেক গভীর হলে, প্রতিটি স্তরে gradient বারবার গুণ হতে হতে ক্রমেই খুব ছোট (প্রায় শূন্য) হয়ে যেতে পারে। ফলে প্রথম দিকের স্তরগুলো প্রায় কিছুই শিখতে পারে না — এই সমস্যা বলা হয় Vanishing Gradient।

৩.২ Exploding Gradient

এর বিপরীত সমস্যাও ঘটতে পারে — gradient বারবার গুণ হতে হতে অত্যন্ত বড় হয়ে যেতে পারে, ফলে প্যারামিটারের মান অস্থির (unstable) হয়ে যায়। একে বলা হয় Exploding Gradient।

৩.৩ Optimization Difficulty (অপ্টিমাইজেশন-জনিত জটিলতা)

নেটওয়ার্ক গভীর হলে loss function-এর ভূপ্রকৃতি (landscape) অনেক জটিল হয়ে যায় — ফলে Gradient Descent-এর মতো অপ্টিমাইজেশন পদ্ধতির পক্ষে ভালো সমাধান (solution) খুঁজে পাওয়া কঠিন হয়ে পড়ে।

✍️ গুরুত্বপূর্ণ পার্থক্য: ResNet গবেষণাপত্রে বর্ণিত Degradation Problem এই সমস্যাগুলোর থেকে ভিন্ন। লেখকরা প্লেইন (plain) নেটওয়ার্কে Batch Normalization ব্যবহার করেই দেখিয়েছেন যে forward-propagated signal-এর variance শূন্য হয়ে যাচ্ছে না, এবং backward gradient-ও স্বাভাবিক (healthy) মাত্রায় আছে। তা সত্ত্বেও গভীর plain network-এর training error বেশি থেকে যাচ্ছে। অর্থাৎ degradation problem-কে সরাসরি vanishing gradient দিয়ে ব্যাখ্যা করা যায় না — এটি মূলত একটি অস্টিমাইজেশন-জনিত সমস্যা, যার প্রকৃত কারণ পরিষ্কার না হলেও এর ফলাফল স্পষ্টভাবে পর্যবেক্ষণযোগ্য (Original Paper, Section ১)।

অংশ ৪: Degradation Problem

এটি পুরো গবেষণাপত্রের সবচেয়ে গুরুত্বপূর্ণ পর্যবেক্ষণ, যা থেকেই Residual Learning-এর ধারণাটি জন্ম নিয়েছে।

8.১ Plain Network বনাম Residual Network

Plain Network

একটি সাধারণ (plain) নেটওয়ার্কে স্তরগুলো একের পর এক সরাসরি স্তুপ (stack) করা থাকে — কোনো shortcut বা bypass path ছাড়াই:

Input → Layer → Layer → Layer → ... → Output

Residual Network

Residual Network-এ কিছু স্তর অতিক্রম করে সরাসরি একটি shortcut connection যোগ করা হয়, যা স্তরগুলোর output-এর সাথে input যোগ করে দেয়:

Input → Residual Layers → Addition (+ input) → Output

8.২ বিস্ময়কর পর্যবেক্ষণ (Surprising Observation)

সাধারণ প্রত্যাশা ছিল, নেটওয়ার্ক গভীর হলে training error কমবে বা অন্তত একই থাকবে। কিন্তু গবেষকরা লক্ষ করেন —

✍️ একটি গভীরতর (deeper) plain network-এর training error একটি অপেক্ষাকৃত অগভীর (shallower) plain network-এর তুলনায় বেশি হতে পারে (Original Paper, Section ১; Figure ১)।

এখানে গুরুত্বপূর্ণ বিষয় হলো — এটি সাধারণ overfitting নয়। Overfitting হলে training error কম থাকত কিন্তু validation/test error বেশি হতো। কিন্তু এখানে training error নিজেই বেশি — অর্থাৎ গভীর নেটওয়ার্কটি প্রশিক্ষণ ডেটাতেই ভালোভাবে fit করতে ব্যর্থ হচ্ছে। সুতরাং এটি একটি অস্টিমাইজেশন-জনিত সমস্যা, data-জনিত সমস্যা নয়।

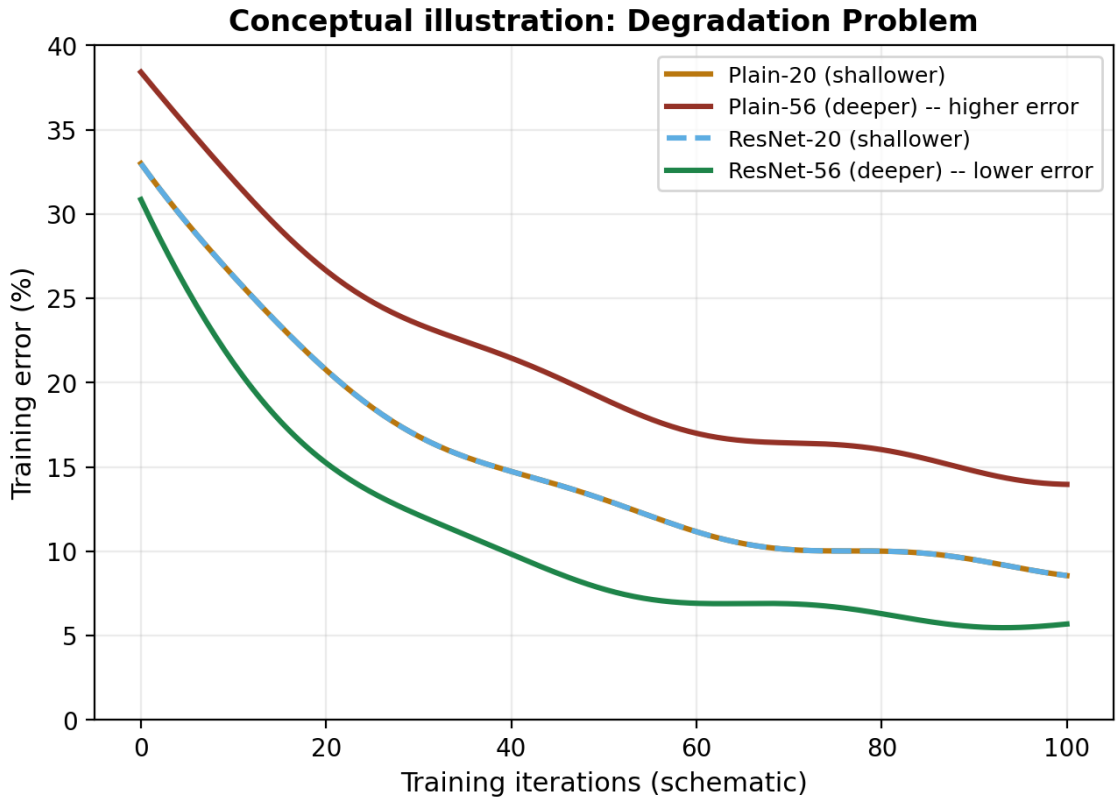
8.৩ Identity-Mapping যুক্তি (Identity Mapping Argument)

লেখকরা একটি যৌক্তিক পরীক্ষা (thought experiment) দিয়ে দেখান কেন এই ফলাফলটি অস্বাভাবিক:

1. ধরা যাক, একটি অপেক্ষাকৃত অগভীর (shallow) নেটওয়ার্ক ইতিমধ্যেই ভালো ফলাফল দিচ্ছে।
2. এখন যদি এই নেটওয়ার্কের সাথে আরও কিছু স্তর যোগ করে একটি গভীরতর নেটওয়ার্ক তৈরি করা হয়, তাহলে তাত্ত্বিকভাবে গভীরতর নেটওয়ার্কটি অন্তত শুরুর অগভীর নেটওয়ার্কের সমান ভালো হওয়া উচিত।
3. কারণ, নতুন যোগ করা অতিরিক্ত স্তরগুলো কেবল Identity Mapping (অর্থাৎ input-কে অবিকৃতভাবে output-এ পাঠানো, $x \rightarrow x$) শিখলেই যথেষ্ট — তাহলে গভীর নেটওয়ার্কটি হুবহু অগভীর নেটওয়ার্কের অনুরূপ আচরণ করবে।
4. সুতরাং, তাত্ত্বিকভাবে গভীরতর নেটওয়ার্কের training error কখনো শুরুর অগভীর নেটওয়ার্কের চেয়ে বেশি হওয়া উচিত নয়।
5. কিন্তু বাস্তব পরীক্ষায় (experiment) দেখা যায়, plain নেটওয়ার্কে এই তাত্ত্বিক প্রত্যাশা মিলছে না — গভীরতর plain network-এর training error বেশি থেকে যাচ্ছে।

এর অর্থ হলো, সাধারণ (unreferenced) optimizer দিয়ে সহজ Identity Mapping শেখানোও কঠিন হয়ে পড়ছে। এই পর্যবেক্ষণটিই লেখকদের একটি নতুন প্রশ্নের দিকে নিয়ে যায় — “যদি Identity Mapping শেখা optimizer-এর জন্য কঠিন হয়, তাহলে optimizer-কে সহজ করে সেই কাজটি এমনভাবে সাজানো যায় কি, যাতে Identity Mapping শেখা সহজাতভাবেই সহজ হয়ে যায়?” এই প্রশ্নের উত্তর থেকেই Residual Learning-এর জন্ম, যা পরের অংশে ব্যাখ্যা করা হয়েছে।

🔍 Original Paper, Section 1; Figure 1



চিত্র 1: Plain vs Residual Network — গভীরতা বনাম training/test error (degradation problem)

অংশ ৫: Residual Learning (রেসিডুয়াল লার্নিং)

এটি সম্পূর্ণ গবেষণাপত্রের কেন্দ্রীয় ধারণা। ধরা যাক, একটি নেটওয়ার্ক স্তর (বা স্তরসমূহ) যে প্রকৃত ম্যাপিং (mapping) শিখতে চাইছে, তা হলো:

$$H(x)$$

প্রথাগত পদ্ধতিতে নেটওয়ার্ক সরাসরি $H(x)$ শেখার চেষ্টা করে। কিন্তু ResNet-এর মূল প্রস্তাব হলো — নেটওয়ার্ককে সরাসরি $H(x)$ শেখানোর বদলে, নিচের residual function $F(x)$ শেখানো হয়:

$$F(x) = H(x) - x$$

তাহলে মূল কাঙ্ক্ষিত ম্যাপিং $H(x)$ -কে আবার নিচের আকারে লেখা যায়:

$$H(x) = F(x) + x$$

Residual Learning বলতে এমন একটি পদ্ধতিকে বোঝায়, যেখানে network সরাসরি desired mapping $H(x)$ শেখার পরিবর্তে residual function $F(x)$ শেখে, এবং শেষে শুধু input x যোগ করে $H(x)$ পুনর্গঠন করে।

৫.১ সংখ্যাগত উদাহরণ (Numerical Example)

ধরা যাক input মান $x = 10$, এবং নেটওয়ার্কের কাঙ্ক্ষিত output $H(x) = 12$ ।

প্রথাগত পদ্ধতিতে নেটওয়ার্ককে সরাসরি সম্পূর্ণ রূপান্তর $10 \rightarrow 12$ শিখতে হতো। কিন্তু Residual Learning-এ:

$$F(x) = H(x) - x = 12 - 10 = 2$$

অর্থাৎ residual branch-কে কেবল +2 পরিমাণ ছোট্ট একটি সংশোধন (correction) শিখলেই চলে — সম্পূর্ণ রূপান্তরটি নয়। এই ছোট্ট সংশোধন শেখা, সম্পূর্ণ নতুন রূপান্তর শেখার চেয়ে সহজ, বিশেষত যখন কাঙ্ক্ষিত ম্যাপিং Identity Mapping-এর কাছাকাছি হয় (অর্থাৎ $H(x) \approx x$)।

☞ যদি কোনো নির্দিষ্ট স্তরের জন্য Identity Mapping-ই সর্বোত্তম (optimal) সমাধান হয়, তাহলে residual function $F(x)$ -কে কেবল শূন্যের কাছাকাছি ঠেলে দিলেই যথেষ্ট ($F(x) \rightarrow 0$), যা একাধিক non-linear স্তর দিয়ে সরাসরি Identity Mapping পুনর্গঠনের চেষ্টা করার চেয়ে অনেক সহজ একটি অপ্টিমাইজেশন লক্ষ্য (Original Paper, Section 3.1)।

অংশ ৬: Residual Block

Residual Learning-এর ধারণাকে বাস্তবে রূপ দেওয়ার একক (unit)-কে বলা হয় Residual Block। একটি মৌলিক (basic) দুই-স্তরবিশিষ্ট Residual Block-এর গঠন নিচে দেখানো হলো:

$$x \rightarrow [3 \times 3 \text{ Conv} \rightarrow \text{ReLU} \rightarrow 3 \times 3 \text{ Conv}] = F(x) \quad ; \quad \text{Output} = \text{ReLU}(F(x) + x)$$

এই ব্লকের প্রতিটি অংশের কাজ:

- Input x : ব্লকে প্রবেশ করা feature
- Main path / Residual branch: দুইটি 3×3 Convolution, মাঝে একটি ReLU — এই পথ residual function $F(x)$ গণনা করে
- Shortcut connection: input x সরাসরি (কোনো রূপান্তর ছাড়াই) main path-কে অতিক্রম (bypass) করে যায়
- Addition: main path-এর output $F(x)$ -এর সাথে shortcut-এর x element-wise যোগ করা হয়
- শেষ ReLU: যোগফলের উপর প্রয়োগ করা চূড়ান্ত non-linearity

এভাবে সম্পূর্ণ ব্লকটি নিচের সমীকরণ মেনে চলে:

$$y = F(x) + x$$

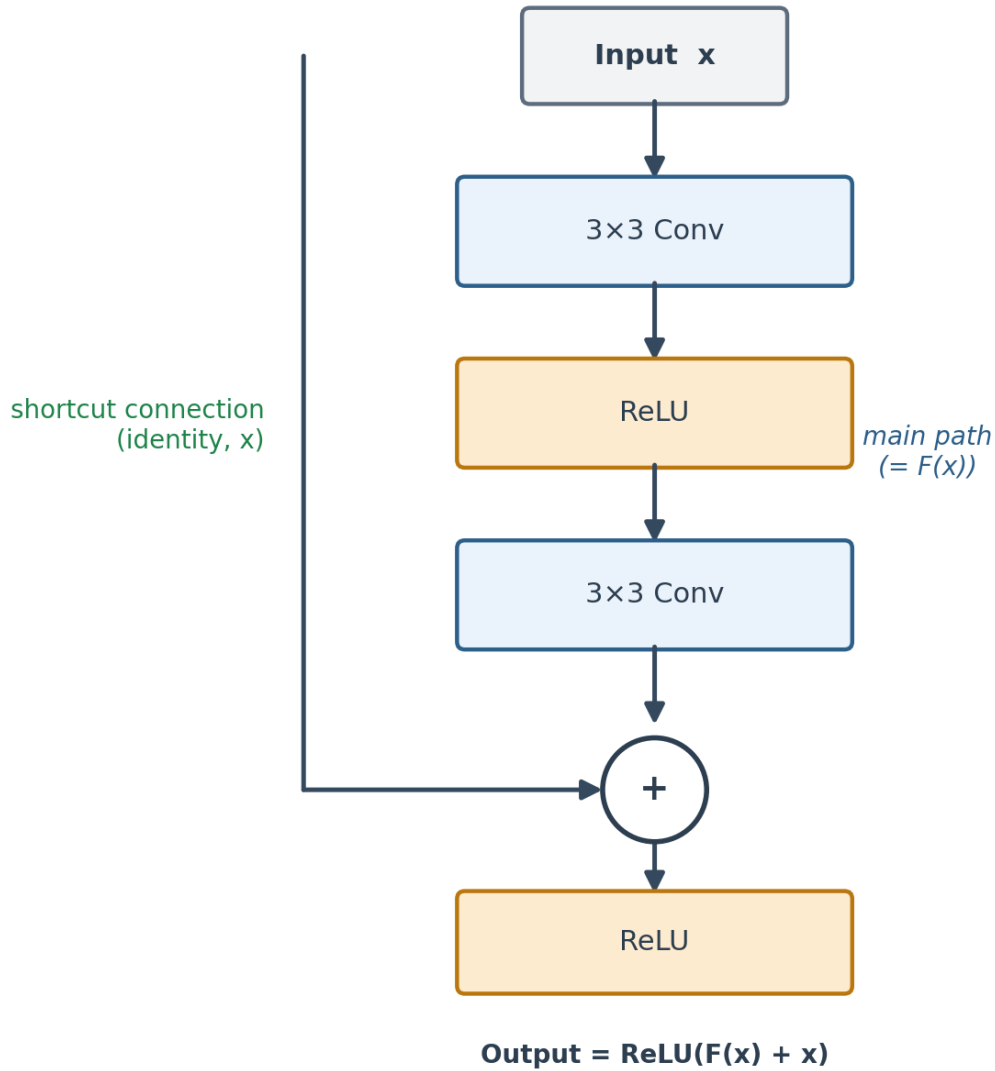
এখানে $F(x)$ হলো ঐ ব্লকের ভেতরের convolution স্তরগুলো (weight W_1, W_2 সহ) যা শিখেছে। দুই-স্তরবিশিষ্ট একটি সাধারণ residual function-কে লেখা যায়:

$$F(x) = W_2 \sigma(W_1 x)$$

ধাপে ধাপে এই গণনা এভাবে ঘটে:

1. x — ইনপুট feature
2. $W_1 x$ — প্রথম convolution স্তর (weight matrix W_1) x -এর উপর প্রয়োগ
3. $\sigma(W_1 x)$ — ReLU activation প্রয়োগ
4. $W_2 \sigma(W_1 x)$ — দ্বিতীয় convolution স্তর (weight matrix W_2) প্রয়োগ করে চূড়ান্ত $F(x)$ পাওয়া
5. $F(x) + x$ — শেষে shortcut-এর মাধ্যমে আসা মূল input x -এর সাথে যোগ

 Original Paper, Figure 2; Section 3.2



চিত্র 2: Residual Block-এর গঠন — main path (3×3 Conv $\rightarrow$ ReLU $\rightarrow 3 \times 3$ Conv) এবং shortcut connection

অংশ ৭: Element-wise Addition (উপাদানভিত্তিক যোগফল)

Element-wise Addition মানে দুটি একই আকারের গ্রিড বা vector-এর সংশ্লিষ্ট (corresponding) অবস্থানের মানগুলো একে অপরের সাথে যোগ করা। উদাহরণ:

$$F(x) = [1, 2, 3] \quad x = [10, 20, 30]$$

$$F(x) + x = [1+10, 2+20, 3+30] = [11, 22, 33]$$

CNN feature map-এর ক্ষেত্রে এই একই নীতি প্রযোজ্য হয় — তবে এখানে প্রতিটি স্থানিক অবস্থান (spatial position, অর্থাৎ height ও width-এর প্রতিটি বিন্দু) এবং প্রতিটি channel আলাদাভাবে যোগ হয়। অর্থাৎ $F(x)$ -এর (h, w, c) অবস্থানের মান x -এর ঠিক একই (h, w, c) অবস্থানের মানের সাথে যোগ হয়।

✎ Element-wise addition করতে হলে $F(x)$ এবং x -এর height, width ও channel — এই তিনটি dimension-ই হুবহু একই হতে হবে; নাহলে যোগফল সংজ্ঞায়িতই (defined) নয়। এই শর্তটিই পরবর্তী অংশে আলোচিত Dimension Mismatch সমস্যার মূল কারণ।

অংশ ৮: Identity Shortcut

Residual Block-এর সমীকরণে —

$$y = F(x) + x$$

এখানে x -কে বলা হয় Identity Mapping, কারণ শর্টকাট পথে x কোনো রূপান্তর (transformation) ছাড়াই সরাসরি পরবর্তী ধাপে পৌঁছে যায়:

$$x \rightarrow x$$

অর্থাৎ, তথ্য (information) কোনো ওজন (weight) বা non-linearity দ্বারা পরিবর্তিত না হয়েই সরাসরি প্রবাহিত (flow) হয়। Identity shortcut-এর গুরুত্বপূর্ণ বৈশিষ্ট্য:

- কোনো নতুন শেখার যোগ্য (trainable) প্যারামিটার যোগ হয় না — shortcut path সম্পূর্ণ parameter-free
- প্রায় কোনো অতিরিক্ত গণনা (computation) প্রয়োজন হয় না
- প্রথম দিকের স্তর থেকে শেষ স্তর পর্যন্ত সরাসরি তথ্য ও gradient প্রবাহের পথ তৈরি হয়, যা গভীর নেটওয়ার্কে অপ্টিমাইজেশনকে সহজ করে

🔍 Original Paper, Section 3.2

অংশ ৯: Dimension Mismatch (মাত্রার অমিল)

সবসময় $F(x)$ এবং x -এর dimension এক নাও হতে পারে — বিশেষত যখন কোনো stage-এর শুরুতে stride ব্যবহার করে স্থানিক আকার (height, width) ছোট করা হয় অথবা channel সংখ্যা বাড়ানো হয়। উদাহরণ:

	Height × Width × Channel
Input x	$56 \times 56 \times 64$
Residual branch output $F(x)$	$28 \times 28 \times 128$

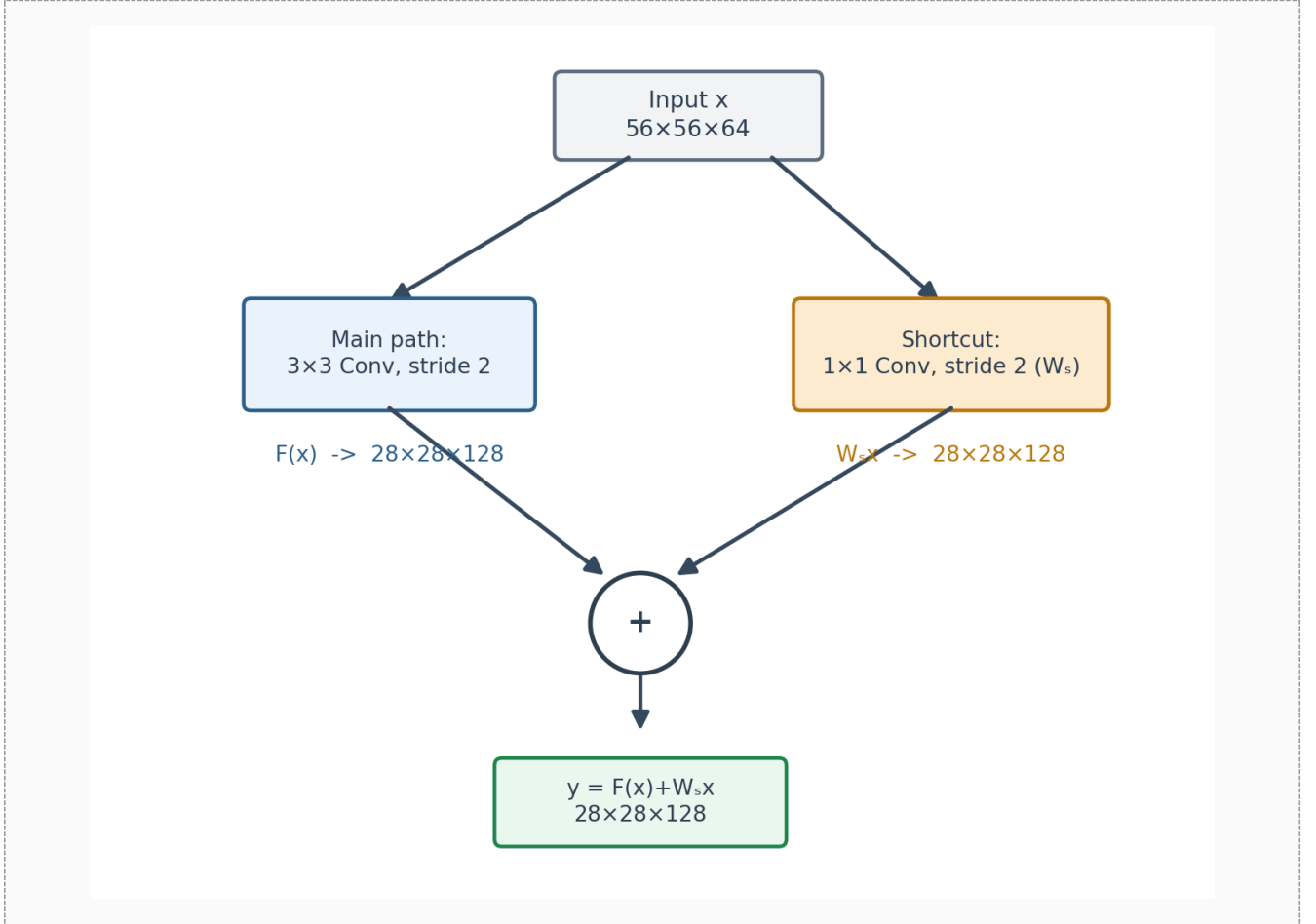
এই দুটি সরাসরি element-wise যোগ করা যায় না, কারণ height, width এবং channel — কোনোটিই মিলছে না। এই সমস্যা সমাধানের জন্য paper একটি projection shortcut প্রস্তাব করে:

$$y = F(x) + W_s x$$

এখানে W_s হলো একটি projection matrix (বাস্তবে একটি 1×1 convolution), যার একমাত্র কাজ হলো x -এর dimension-কে $F(x)$ -এর dimension-এর সাথে মিলিয়ে দেওয়া — এটি কোনো জটিল feature শেখে না, শুধু আকার মেলায়। ব্যবহারিকভাবে:

$$56 \times 56 \times 64 \rightarrow 1 \times 1 \text{ Conv, stride 2} \rightarrow 28 \times 28 \times 128$$

এখন $W_s x$ এবং $F(x)$ — উভয়ের dimension $28 \times 28 \times 128$ হওয়ায় এদের element-wise যোগফল সংজ্ঞায়িত এবং গণনাযোগ্য।



চিত্র 3: Dimension Mismatch সমস্যা এবং 1×1 Conv দিয়ে সমাধান (projection shortcut W_s)

অংশ ১০: 1×1 Convolution

1×1 Convolution দেখতে অত্যন্ত সরল মনে হলেও এটি আধুনিক CNN স্থাপত্যের (architecture) একটি অত্যন্ত গুরুত্বপূর্ণ ও শক্তিশালী উপাদান। এটি স্থানিক (spatial) কোনো প্যাটার্ন দেখে না (কারণ kernel-এর আকার মাত্র ১×১), কিন্তু এটি channel-জুড়ে (across channels) তথ্য মিশ্রিত ও রূপান্তরিত করে।

1×1 Convolution-এর প্রধান ব্যবহার:

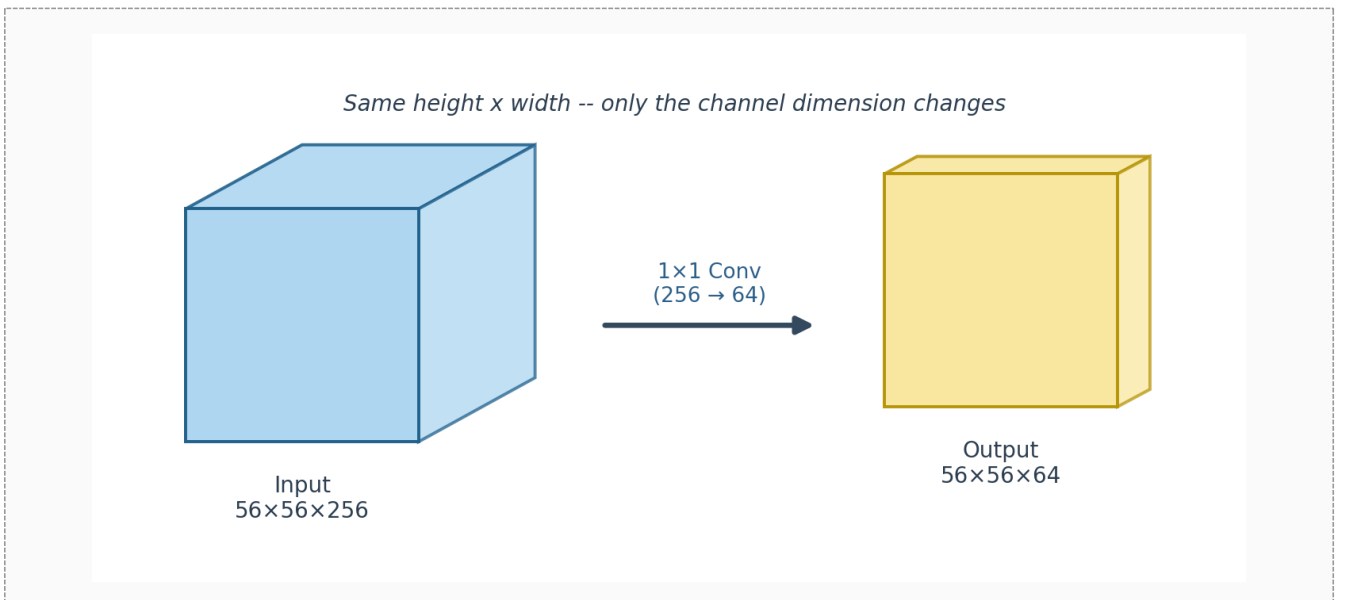
1. Channel সংখ্যা পরিবর্তন করা
2. Channel সংখ্যা কমানো (dimensionality reduction)
3. Channel সংখ্যা বাড়ানো
4. গণনার পরিমাণ (computation) কমানো
5. Dimension মেলানো (projection shortcut-এ ব্যবহৃত হয়)
6. Bottleneck Block তৈরি করা

উদাহরণ:

Input	Operation	Output
$56 \times 56 \times 256$	1×1 Conv (256 → 64)	$56 \times 56 \times 64$

স্ট্রাইড (stride) ১ হলে height ও width অপরিবর্তিত থাকে, কারণ kernel-এর আকার ১×১ হওয়ায় এটি ছবির উপর দিয়ে কোনো স্থানিক অঞ্চল (region) না দেখে শুধু একটিমাত্র pixel অবস্থানের সবগুলো channel দেখে।

গাণিতিকভাবে, একটি নির্দিষ্ট স্থানিক অবস্থান (একটি pixel) বিবেচনা করলে, সেই অবস্থানে input-এর ২৫৬টি channel-এর মান একটি vector গঠন করে। 1×1 convolution আসলে সেই ২৫৬-মাত্রিক vector-এর উপর একটি সরল রৈখিক রূপান্তর (linear transformation) প্রয়োগ করে ৬৪-মাত্রিক নতুন vector তৈরি করে — এটি প্রতিটি স্থানিক অবস্থানের জন্য পুনরাবৃত্তি হয়। এভাবে height ও width অক্ষত রেখে কেবল channel dimension-এ রূপান্তর ঘটে।



চিত্র 4: 1×1 Convolution — 256 channel থেকে 64 channel-এ রূপান্তর

অংশ ১১: Bottleneck Block

ResNet-50, ResNet-101 এবং ResNet-152-এ ব্যবহৃত হয় Bottleneck Block, যা তিনটি convolution স্তরের সমন্বয়ে গঠিত:

Input (256 ch) $\rightarrow$ 1×1 Conv (256 $\rightarrow$ 64) $\rightarrow$ 3×3 Conv (64 $\rightarrow$ 64) $\rightarrow$ 1×1 Conv (64 $\rightarrow$ 256) $\rightarrow$ Add Shortcut $\rightarrow$ Output (256 ch)

প্রতিটি ধাপ ব্যাখ্যা করা যাক:

- প্রথম 1×1 Conv (256 $\rightarrow$ 64): channel সংখ্যা কমিয়ে দেয় — এটিই 'bottleneck' বা সংকীর্ণ পথ তৈরি করে।
- মাঝের 3×3 Conv (64 $\rightarrow$ 64): মূল স্থানিক (spatial) বৈশিষ্ট্য নিষ্কাশন এই স্তরেই ঘটে — কিন্তু এটি চালানো হয় অনেক কম (৬৪টি) channel-এর উপর।
- শেষ 1×1 Conv (64 $\rightarrow$ 256): channel সংখ্যা আবার বাড়িয়ে মূল আকারে (২৫৬) ফিরিয়ে আনে, যাতে shortcut-এর সাথে যোগ করা যায়।
- Shortcut যোগ: মূল input-কে (প্রয়োজনে projection সহ) ব্লকের output-এর সাথে যোগ করা হয়।

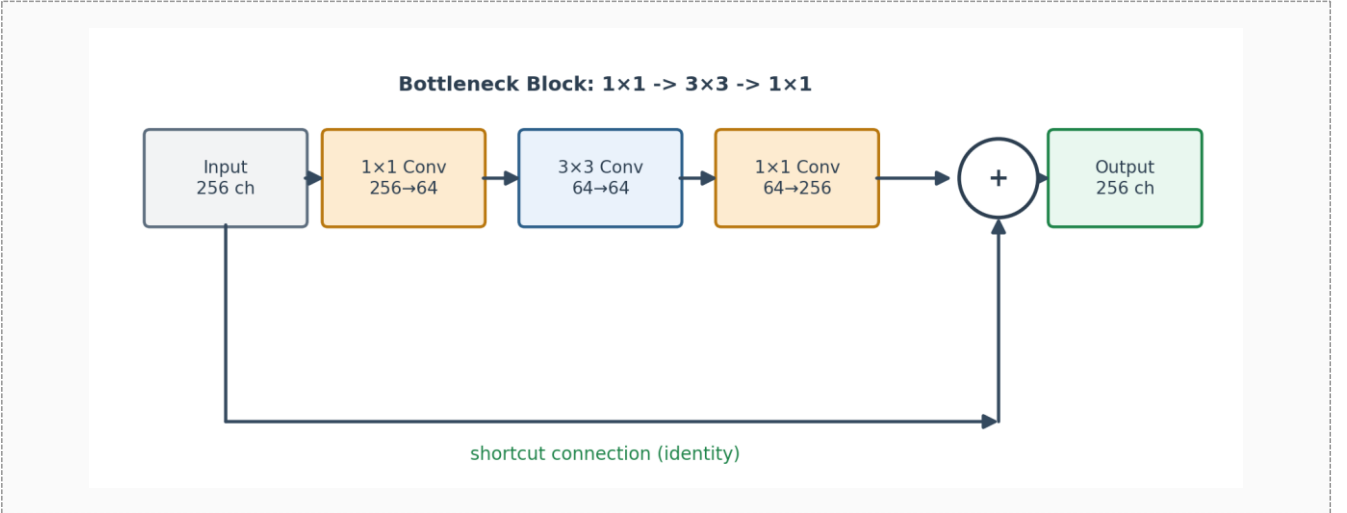
১১.১ কেন 256 $\rightarrow$ 64 $\rightarrow$ 64 $\rightarrow$ 256?

যদি 3×3 convolution সরাসরি ২৫৬ channel-এর উপর প্রয়োগ করা হতো, তাহলে তার গণনার পরিমাণ (parameter ও FLOPs) অনেক বেশি হতো, কারণ 3×3 convolution-এর খরচ input ও output channel সংখ্যার গুণফলের সমানুপাতিক। প্রথমে 1×1 Conv দিয়ে channel সংখ্যা ২৫৬ থেকে ৬৪-এ নামিয়ে আনলে, ব্যয়বহুল 3×3 convolution অনেক কম (৬৪টি) channel-এর উপর চালানো যায় — এতে গণনা অনেক কমে যায়, কিন্তু representational ক্ষমতা তেমন কমে না। শেষে আবার 1×1 Conv দিয়ে channel সংখ্যা ২৫৬-এ ফিরিয়ে আনা হয়, যাতে shortcut-এর সাথে dimension মেলে এবং পরবর্তী ব্লকের জন্য পর্যাপ্ত representational ক্ষমতা বজায় থাকে।

এই আকৃতির কারণেই একে 'Bottleneck' (সংকীর্ণ পথ) বলা হয় — একটি বোতলের গলার মতো, যেখানে মাঝখানে channel সংখ্যা সংকুচিত হয়ে যায়, তারপর আবার প্রশস্ত হয়। এটিকে অনেকটা একটি ঘণ্টা-আকৃতির (hourglass) পথের সাথে তুলনা করা যায় — শুরুতে চওড়া, মাঝে সরু, শেষে আবার চওড়া।

১১.২ Basic Block বনাম Bottleneck Block — তুলনা

বৈশিষ্ট্য	Basic Residual Block (ResNet-18/34)	Bottleneck Block (ResNet-50/101/152)
গঠন	$3\times 3 \rightarrow 3\times 3$	$1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$
স্তরের সংখ্যা	২টি convolution	৩টি convolution
গণনার খরচ	তুলনামূলক বেশি (একই channel-এ 3×3 প্রয়োগ)	তুলনামূলক কম (3×3 কেবল সংকুচিত channel-এ প্রয়োগ)
মূল উদ্দেশ্য	অগভীর/মাঝারি গভীরতার নেটওয়ার্কে সরল ব্লক	অত্যন্ত গভীর নেটওয়ার্কে গণনাগতভাবে ব্যবহারযোগ্য রাখা

চিত্র 5: Bottleneck Block গঠন: $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$, শর্টকাট শাখাসহ

অংশ ১২: Global Average Pooling (GAP)

নেটওয়ার্কের শেষ convolution stage-এর পর ResNet একটি Global Average Pooling ব্যবহার করে, যা প্রতিটি channel-এর সম্পূর্ণ স্থানিক (spatial) অংশের গড় (average) নিয়ে একটিমাত্র সংখ্যায় পরিণত করে:

$$7 \times 7 \times 2048 \rightarrow \text{Global Average Pooling} \rightarrow 1 \times 1 \times 2048$$

সংখ্যাগত উদাহরণ — একটি 2×2 feature map:

1	2
3	4

$$\text{Average} = (1+2+3+4)/4 = 2.5$$

প্রতিটি channel-এর জন্য এই গড় আলাদাভাবে গণনা করা হয়। এভাবে $7 \times 7 \times 2048$ আকারের feature map একটিমাত্র 2048-মাত্রিক vector-এ পরিণত হয়, যাকে বলা যায় ছবির একটি সংক্ষিপ্ত 'সারাংশ' (summary) representation।

১২.১ শ্রেণিবিন্যাসের (Classification) শেষ ধাপ

$$7 \times 7 \times 2048 \rightarrow \text{GAP} \rightarrow 1 \times 1 \times 2048 \rightarrow \text{Flatten} \rightarrow 2048 \rightarrow \text{Fully Connected} \rightarrow 1000 \text{ classes} \rightarrow \text{Softmax}$$

VGG-এর মতো আগের স্থাপত্যে classification-এর শেষে বড় আকারের (কোটি কোটি প্যারামিটারযুক্ত) Fully Connected স্তর ব্যবহৃত হতো, যা অনেক বেশি প্যারামিটার যোগ করত এবং overfitting-এর ঝুঁকি

বাড়াত। বিপরীতে, Global Average Pooling কোনো প্যারামিটার ছাড়াই feature map-কে সরাসরি একটি vector-এ পরিণত করে — ফলে চূড়ান্ত Fully Connected স্তরটি অনেক ছোট ও সহজ (মাত্র 2048 → 1000) হয়ে যায়, প্যারামিটার সংখ্যা কমে যায়, এবং overfitting-এর সম্ভাবনাও কমে।

অংশ ১৩: ResNet-এর বিভিন্ন স্থাপত্য (Architectures)

মূল গবেষণাপত্রে পাঁচটি প্রধান ResNet স্থাপত্য উপস্থাপন করা হয়েছে — ResNet-18, 34, 50, 101 এবং 152। নিচে তাদের তুলনামূলক সারণি দেওয়া হলো (Original Paper, Table 1 অনুযায়ী):

Model	মোট স্তর	Block Type	প্রতি stage-এ ব্লক সংখ্যা (conv2-conv5)	FLOPs
ResNet-18	18	Basic ($3\times3\rightarrow3\times3$)	2, 2, 2, 2	1.8×10^9
ResNet-34	34	Basic ($3\times3\rightarrow3\times3$)	3, 4, 6, 3	3.6×10^9
ResNet-50	50	Bottleneck ($1\times1\rightarrow3\times3\rightarrow1\times1$)	3, 4, 6, 3	3.8×10^9
ResNet-101	101	Bottleneck ($1\times1\rightarrow3\times3\rightarrow1\times1$)	3, 4, 23, 3	7.6×10^9
ResNet-152	152	Bottleneck ($1\times1\rightarrow3\times3\rightarrow1\times1$)	3, 8, 36, 3	11.3×10^9

লক্ষণীয়:

- ResNet-18/34 ব্যবহার করে Basic Block: $3\times3 \rightarrow 3\times3$
- ResNet-50/101/152 ব্যবহার করে Bottleneck Block: $1\times1 \rightarrow 3\times3 \rightarrow 1\times1$

Bottleneck design-এর কারণেই ResNet-50 এবং ResNet-34 প্রায় একই পরিমাণ FLOPs (3.6×10^9 বনাম 3.8×10^9) ব্যবহার করেও ResNet-50-এ একই সংখ্যক ব্লকে অনেক বেশি (৫০টি) স্তর সমাহিত করা সম্ভব হয়েছে। এভাবে bottleneck design নেটওয়ার্ককে গণনাগতভাবে ব্যবহারযোগ্য (practical) রেখেই আরও গভীর করার সুযোগ দিয়েছে।

 Original Paper, Table 1

অংশ ১৪: ResNet-50 স্থাপত্য — ধাপে ধাপে

ResNet-50-এর সম্পূর্ণ পাইপলাইন (Input থেকে Softmax পর্যন্ত) নিচে ধাপে ধাপে দেখানো হলো:

Input $\rightarrow 7\times7$ Conv, 64, stride 2 $\rightarrow 3\times3$ MaxPool, stride 2 $\rightarrow$ Conv2_x $\rightarrow$ Conv3_x $\rightarrow$ Conv4_x $\rightarrow$ Conv5_x $\rightarrow$ GAP $\rightarrow$ FC $\rightarrow$ Softmax

১৪.১ স্থানিক আকারের পরিবর্তন (Spatial Dimension Progression)

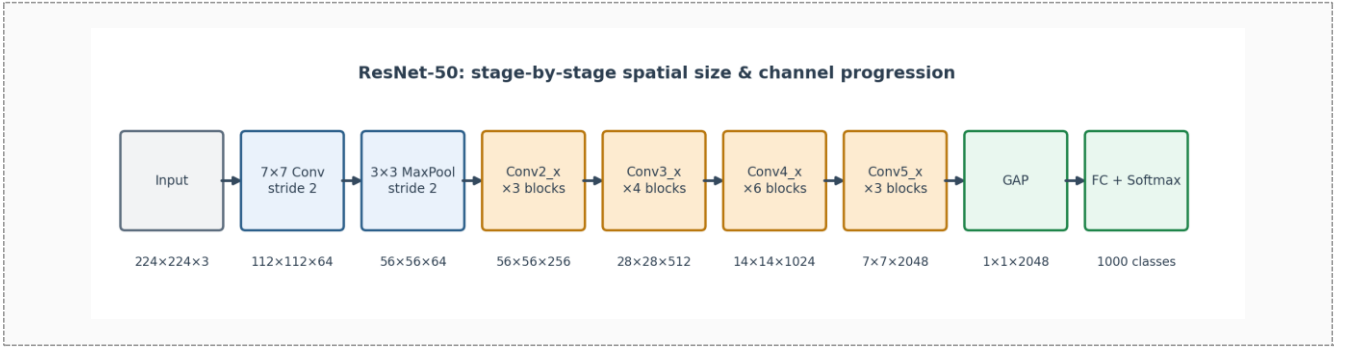
$224\times224 \rightarrow 112\times112 \rightarrow 56\times56 \rightarrow 28\times28 \rightarrow 14\times14 \rightarrow 7\times7 \rightarrow 1\times1$

১৪.২ Channel সংখ্যার পরিবর্তন (Channel Progression)

$$224 \times 224 \times 3 \rightarrow 112 \times 112 \times 64 \rightarrow 56 \times 56 \times 256 \rightarrow 28 \times 28 \times 512 \rightarrow 14 \times 14 \times 1024 \rightarrow 7 \times 7 \times 2048$$

এই দুটি প্রবণতা লক্ষ করলে একটি স্পষ্ট নকশা-নীতি (design principle) বোঝা যায়:

- স্থানিক আকার (height $\times$ width) ক্রমান্বয়ে কমে (downsampling) — প্রতিটি নতুন stage-এর শুরুতে stride 2 ব্যবহার করে
- Channel সংখ্যা একই সাথে ক্রমান্বয়ে বাড়ে — প্রতিটি নতুন stage-এ channel সংখ্যা দ্বিগুণ হয় এর পেছনের যুক্তি হলো — স্থানিক তথ্য (spatial detail) ধীরে ধীরে কমিয়ে ফেলার সাথে সাথে, ঐ তথ্যকে আরও বেশি বিমূর্ত ও সমৃদ্ধ (richer, more abstract) representation-এ (বেশি channel-এর মাধ্যমে) রূপান্তরিত করা হয়। এভাবে মোট গণনার পরিমাণ প্রতিটি stage-এ প্রায় সমান রাখা সম্ভব হয়।



চিত্র 6: ResNet-50 সম্পূর্ণ স্থাপত্য — stage অনুযায়ী স্থানিক আকার ও channel পরিবর্তন

অংশ ১৫: মূল গবেষণাপত্রের Table 1 বোঝা

গবেষণাপত্রের Table 1-এ প্রতিটি stage-এর গঠন সংক্ষিপ্ত আকারে এভাবে লেখা হয়:

$$[1 \times 1, 64 / 3 \times 3, 64 / 1 \times 1, 256] \times 3$$

এখানে ব্র্যাকেটের ভেতরের অংশ একটি সম্পূর্ণ Bottleneck Block বোঝায় (1×1 Conv দিয়ে 64 channel, তারপর 3×3 Conv 64 channel, তারপর 1×1 Conv দিয়ে 256 channel-এ ফেরত)। আর ব্র্যাকেটের বাইরে ' $\times 3$ ' মানে হলো — এই সম্পূর্ণ ব্লকটি ছব্ব ৩ বার পরপর স্তুপ (stack) করা হয়েছে, প্রতিটি ব্লকের output পরের ব্লকের input হিসেবে ব্যবহৃত হয়।

ResNet-50-এর জন্য প্রতিটি stage-এ ব্লক-পুনরাবৃত্তির সংখ্যা:

- Conv2_x $\times 3$
- Conv3_x $\times 4$
- Conv4_x $\times 6$
- Conv5_x $\times 3$

১৫.১ কীভাবে মোট ৫০টি স্তর গণনা করা হয়

প্রতিটি Bottleneck Block-এ ৩টি করে convolution স্তর থাকে। তাই মোট ব্লক-সংখ্যা এবং স্তর-সংখ্যা ধাপে ধাপে গণনা করা যাক:

1. মোট ব্লক = $3 + 4 + 6 + 3 = 16$ টি Bottleneck Block
2. প্রতিটি ব্লকে ৩টি convolution স্তর $\rightarrow 16 \times 3 = 48$ টি convolution স্তর
3. শুরুতে ১টি 7×7 convolution স্তর (conv1)
4. শেষে ১টি Fully Connected স্তর
5. মোট = $48 + 1 + 1 = 50$ স্তর — এখান থেকেই নামকরণ 'ResNet-50'

একইভাবে ResNet-101 (block: 3, 4, 23, 3 $\rightarrow$ মোট 33 ব্লক $\times 3 = 99 + 1 + 1 = 101$) এবং ResNet-152 (block: 3, 8, 36, 3 $\rightarrow$ মোট 50 ব্লক $\times 3 = 150 + 1 + 1 = 152$)-এর স্তর সংখ্যাও একইভাবে গণনা করা যায়। ResNet-18/34-এ Basic Block ব্যবহৃত হয়, যাতে প্রতি ব্লকে ২টি করে convolution স্তর থাকে (তাই ResNet-34: block 3,4,6,3 = 16 ব্লক $\times 2 = 32 + 1 + 1 = 34$)।

 Original Paper, Table 1

অংশ ১৬: মূল গবেষণাপত্রের প্রশিক্ষণ সেটআপ (Implementation)

গবেষণাপত্রে বর্ণিত মূল প্রশিক্ষণ পদ্ধতি সংক্ষেপে নিচে দেওয়া হলো, প্রতিটির উদ্দেশ্যসহ:

উপাদান	উদ্দেশ্য
Random crop	ছবি থেকে এলোমেলোভাবে অংশ কেটে নেওয়া, যাতে নেটওয়ার্ক বিভিন্ন অংশ ও প্রেক্ষাপট (context) থেকে শিখতে পারে (data augmentation)

উপাদান	উদ্দেশ্য
Horizontal flip	ছবিকে অনুভূমিকভাবে উল্টে দেওয়া, যাতে নেটওয়ার্ক দিক-নিরপেক্ষভাবে (orientation-invariant) শিখতে পারে
Color augmentation	রঙের সামান্য তারতম্য যোগ করা, যাতে নেটওয়ার্ক আলো/রঙের পরিবর্তনের প্রতি বেশি সহনশীল (robust) হয়
Batch Normalization	প্রতিটি convolution-এর পর প্রয়োগ করে প্রশিক্ষণ স্থিতিশীল ও দ্রুত করা
SGD (Stochastic Gradient Descent)	নেটওয়ার্কের weight আপডেট করার মূল অপটিমাইজেশন পদ্ধতি
Mini-batch	একবারে অল্প কিছু ছবি (যেমন ২৫৬টি) নিয়ে weight আপডেট করা, সম্পূর্ণ ডেটাসেট একসাথে নয়
Learning rate	প্রতিবার weight কতটুকু পরিবর্তন হবে তা নিয়ন্ত্রণ করে; প্রশিক্ষণের সময় ধাপে ধাপে কমানো হয়েছে
Momentum	অতীত gradient-এর দিক মনে রেখে অপটিমাইজেশনকে মসৃণ ও দ্রুত করে
Weight decay	weight-এর মান অতিরিক্ত বড় হয়ে যাওয়া রোধ করে (regularization), overfitting কমায়ে
Training iterations	নেটওয়ার্ক মোট কতগুলো ধাপ (iteration) ধরে প্রশিক্ষিত হয়েছে তা নির্ধারণ করে
Dropout	মূল ResNet স্থাপত্যে ব্যবহৃত হয়নি — কারণ Batch Normalization ইতিমধ্যেই যথেষ্ট নিয়মিতকরণ (regularization) প্রভাব প্রদান করে

Original Paper, Section 3.4

অংশ ১৭: ImageNet পরীক্ষা-নিরীক্ষা (ImageNet Experiments)

১৭.১ ImageNet 2012 ডেটাসেট

- মোট শ্রেণি (class): ১০০০টি
- প্রশিক্ষণ ছবি (training images): ১২ লক্ষ ৮০ হাজার (1.28 million)
- যাচাইকরণ ছবি (validation images): ৫০,০০০টি
- পরীক্ষার ছবি (test images): ১,০০,০০০টি

১৭.২ Top-1 ও Top-5 Error

- Top-1 Error: নেটওয়ার্কের সবচেয়ে বেশি সম্ভাবনায়ুক্ত (highest probability) পূর্বাভাসটি যদি সঠিক শ্রেণির সাথে না মেলে, তাহলে তা ভুল হিসেবে গণ্য হয়।
- Top-5 Error: নেটওয়ার্কের প্রথম ৫টি সর্বাধিক সম্ভাবনায়ুক্ত পূর্বাভাসের মধ্যে সঠিক শ্রেণি না থাকলে তবেই তা ভুল হিসেবে গণ্য হয়। এটি Top-1-এর তুলনায় কম কঠোর একটি পরিমাপ (metric)।

১৭.৩ Plain বনাম ResNet — মূল ফলাফল

Original Paper-এর Table 2 অনুযায়ী (Top-1 error %, 10-crop testing):

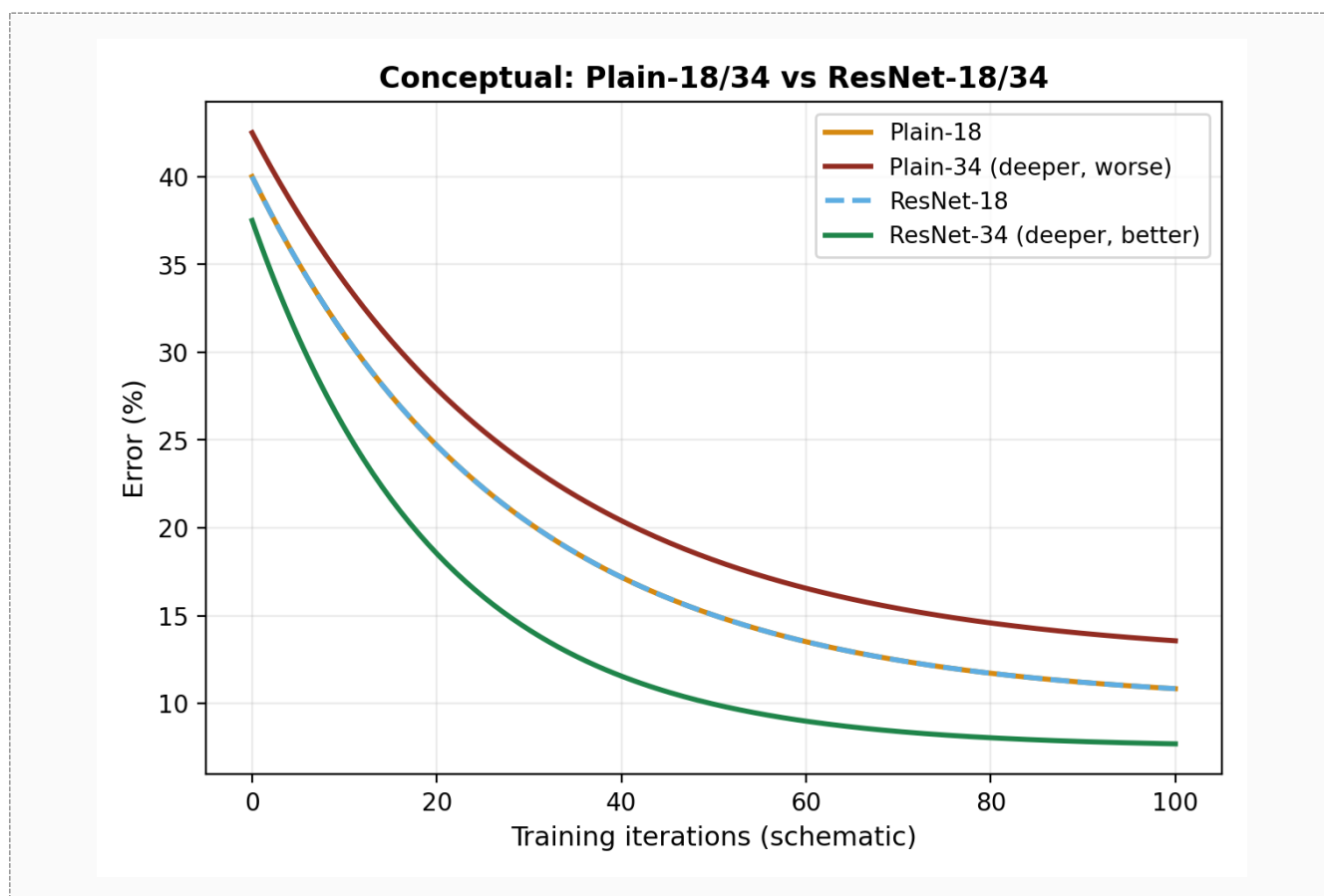
স্তর সংখ্যা	Plain Network	ResNet
18-layer	27.94	27.88
34-layer	28.54	25.03

এই সারণি থেকে দুটি গুরুত্বপূর্ণ পর্যবেক্ষণ পাওয়া যায়:

1. Plain-34-এর error (28.54%) Plain-18-এর error (27.94%)-এর চেয়ে বেশি — অর্থাৎ plain network-এ গভীরতা বৃদ্ধি করলে কর্মক্ষমতা খারাপ হয়েছে, যা degradation problem-এর সরাসরি প্রমাণ।
2. ResNet-34-এর error (25.03%) ResNet-18-এর error (27.88%)-এর চেয়ে অনেক কম — অর্থাৎ residual connection যোগ করার পর গভীরতা বৃদ্ধি করলে কর্মক্ষমতা উন্নত হয়েছে।

এই দুটি পর্যবেক্ষণ একসাথে বিবেচনা করলে বোঝা যায় — সমস্যাটি নেটওয়ার্কের representational ক্ষমতার (capacity) অভাব ছিল না, বরং অপিটমাইজেশনের সমস্যা ছিল। Residual connection যোগ করার ফলে ঠিক একই গভীরতা ও প্যারামিটার সংখ্যা বজায় রেখেও (কারণ shortcut কোনো নতুন প্যারামিটার যোগ করে না) নেটওয়ার্ক অনেক ভালো ফলাফল দিয়েছে। এটিই Residual Learning-এর কার্যকারিতার সবচেয়ে শক্তিশালী পরীক্ষামূলক প্রমাণ।

🔍 Original Paper, Figure 4; Table 2



চিত্র 7: Plain-18/34 বনাম ResNet-18/34 — training ও validation error curve

অংশ ১৮: Identity বনাম Projection Shortcut

Dimension না মিললে shortcut path-কে কীভাবে সামলানো যায়, তার জন্য গবেষকরা তিনটি বিকল্প (option) তুলনা করেছেন:

Option A — Zero-padding Identity Shortcut

যেখানে dimension বাড়ে, সেখানে identity shortcut ব্যবহার করা হয়, কিন্তু বাড়তি channel-গুলো শূন্য (zero) দিয়ে padding করে ভরাট করা হয়। এতে কোনো নতুন প্যারামিটার যোগ হয় না।

Option B — Projection Shortcut শুধু dimension বাড়ার জায়গায়

যেখানে dimension বাড়ে, সেখানে 1×1 convolution (W_s) ব্যবহার করে projection shortcut প্রয়োগ করা হয়; বাকি সব জায়গায় সরল identity shortcut ব্যবহৃত হয়।

Option C — সর্বত্র Projection Shortcut

সব শর্টকাট সংযোগেই (dimension পরিবর্তন হোক বা না হোক) 1×1 convolution projection ব্যবহার করা হয়।

Original Paper-এর Table 3 অনুযায়ী (ResNet-34, ImageNet validation, top-1/top-5 error %, 10-crop testing):

Option	বর্ণনা	Top-1 Error	Top-5 Error
A	Zero-padding shortcut (parameter-free)	25.03	7.76
B	Projection শুধু dimension বৃদ্ধির স্থানে	24.52	7.46
C	সর্বত্র Projection shortcut	24.19	7.40

ফলাফল দেখায়, B এবং C, A-এর চেয়ে সামান্য ভালো — কারণ Option A-এর zero-padded অংশে প্রকৃতপক্ষে কোনো residual learning ঘটে না। তবে B এবং C-এর মধ্যে পার্থক্য খুবই সামান্য, অথচ Option C-তে উল্লেখযোগ্য বেশি প্যারামিটার ও গণনা প্রয়োজন হয়।

এই কারণেই লেখকরা মূলত Option B ব্যবহার করেছেন ResNet-50/101/152-এর জন্য — এটি ভালো নির্ভুলতা এবং যুক্তিসঙ্গত গণনার (computation) মধ্যে সবচেয়ে ভালো ভারসাম্য (trade-off) প্রদান করে। Option C ব্যবহার না করার মূল কারণ হলো এর অতিরিক্ত জটিলতা ও গণনার খরচ, যা প্রান্তিক (marginal) উন্নতির তুলনায় ন্যায্য নয়।

 Original Paper, Section 4.1; Table 3

অংশ ১৯: গভীরতর ResNet-এর ফলাফল

Original Paper-এর Table 3 ও Table 4 অনুযায়ী বিভিন্ন গভীরতার ResNet-এর ImageNet validation ফলাফল (single-model, top-1/top-5 error %):

Model	Top-1 Error	Top-5 Error	FLOPs
ResNet-34 (Option B)	24.52	7.46	3.6×10^9

Model	Top-1 Error	Top-5 Error	FLOPs
ResNet-34 (Option C)	24.19	7.40	3.6×10^9
ResNet-50	22.85	6.71	3.8×10^9
ResNet-101	21.75	6.05	7.6×10^9
ResNet-152	21.43	5.71	11.3×10^9

এছাড়া single-crop (10-crop নয়) মূল্যায়নে ResNet-152 একটি single model হিসেবে 4.49% top-5 error অর্জন করে, যা তৎকালীন যেকোনো একক (single) model-এর চেয়ে ভালো ছিল। পরবর্তীতে বিভিন্ন গভীরতার ৬টি মডেলের একটি ensemble ব্যবহার করে লেখকরা ImageNet test set-এ 3.57% top-5 error অর্জন করেন, যা ILSVRC 2015 প্রতিযোগিতায় শ্রেণিবিন্যাস (classification) বিভাগে প্রথম স্থান অর্জন করে।

মূল উপসংহার:

✍ Plain network-এর মতো ResNet-এ গভীরতা বাড়ানোর সাথে সাথে একই degradation problem দেখা যায়নি— বরং গভীরতা যত বেড়েছে (৩৪ থেকে ১৫২ স্তর পর্যন্ত), error তত ধারাবাহিকভাবে কমেছে। এটি স্পষ্টভাবে প্রমাণ করে যে Residual Learning গভীর নেটওয়ার্ককে অপ্টিমাইজ করা সহজ করে দিয়েছে। এখানে উল্লেখ্য, এই সংখ্যাগুলো ২০১৫ সালে প্রকাশিত মূল গবেষণাপত্রের ঐতিহাসিক (historical) ফলাফল।

🔗 Original Paper, Tables 3 and 4

অংশ ২০: CIFAR-10 পরীক্ষা-নিরীক্ষা

২০.১ CIFAR-10 ডেটাসেট পরিচিতি

- ছবির আকার: 32×32 (তুলনামূলক অনেক ছোট, ImageNet-এর 224×224 -এর চেয়ে)
- মোট শ্রেণি: ১০টি
- প্রশিক্ষণ ছবি: ৫০,০০০টি
- পরীক্ষার ছবি: ১০,০০০টি

CIFAR-10-এ লেখকরা অত্যন্ত গভীর নেটওয়ার্কের আচরণ পর্যবেক্ষণ করার জন্য বিশেষভাবে সরল স্থাপত্য ব্যবহার করে বিভিন্ন গভীরতার ResNet পরীক্ষা করেন: ResNet-20, 32, 44, 56, 110 এবং এমনকি একটি চরম পরীক্ষা হিসেবে ResNet-1202।

২০.২ Original Paper-এর Table 6 (CIFAR-10 test error)

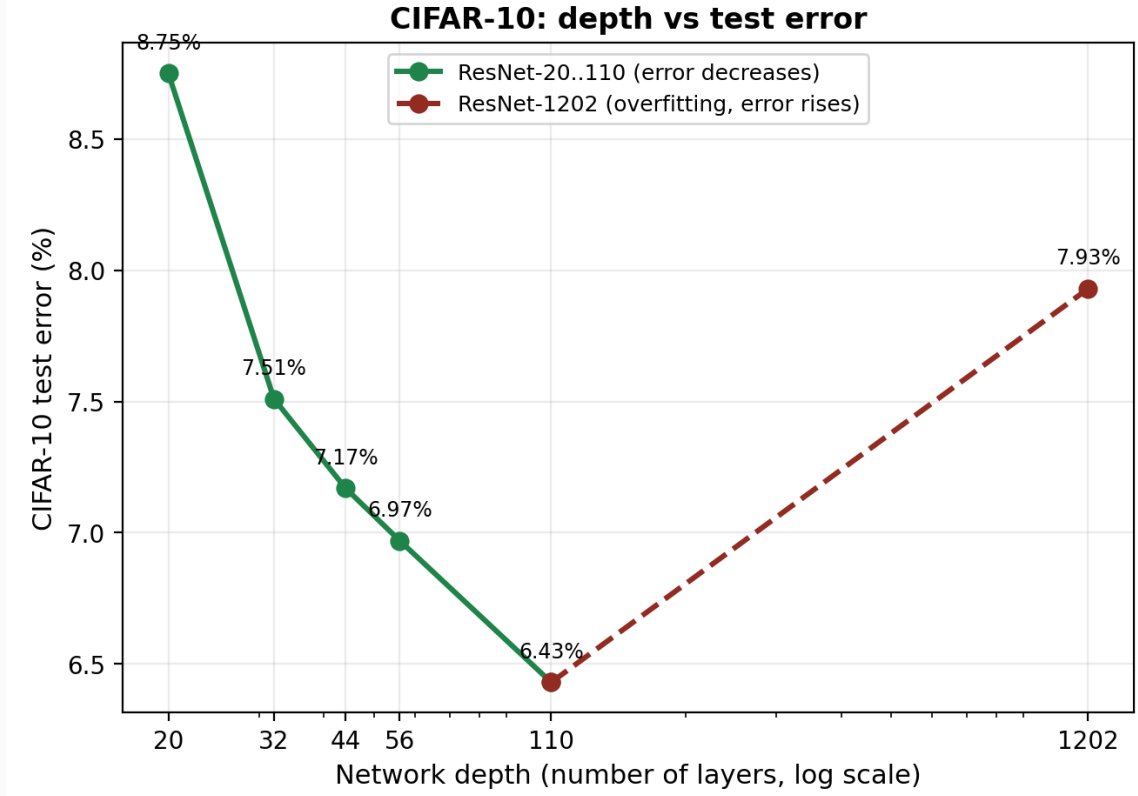
Model	স্তর সংখ্যা	প্যারামিটার সংখ্যা	Test Error (%)
ResNet-20	20	0.27M	8.75
ResNet-32	32	0.46M	7.51
ResNet-44	44	0.66M	7.17
ResNet-56	56	0.85M	6.97
ResNet-110	110	1.7M	6.43 (best of 5 runs)
ResNet-1202	1202	19.4M	7.93

প্রথম পাঁচটি মডেল (২০ থেকে ১১০ স্তর) লক্ষ করলে দেখা যায়, গভীরতা বৃদ্ধির সাথে সাথে test error ধারাবাহিকভাবে কমেছে — অর্থাৎ ResNet-110 পর্যন্ত গভীরতা বাড়ানো সফলভাবে কর্মক্ষমতা উন্নত করেছে, এবং এত গভীর নেটওয়ার্ক (১১০ স্তর) কোনো বড় ধরনের অস্টিমাইজেশন সমস্যা ছাড়াই সফলভাবে প্রশিক্ষিত হয়েছে।

২০.৩ ResNet-1202: অতিরিক্ত গভীরতা ও Overfitting

তবে ResNet-1202 (১২০২ স্তর!)-এর ক্ষেত্রে একটি ভিন্ন চিত্র দেখা যায়। এই নেটওয়ার্কের training error অত্যন্ত কম (প্রায় শূন্যের কাছাকাছি) ছিল, অর্থাৎ এটি প্রশিক্ষণ ডেটা প্রায় নিখুঁতভাবে শিখে ফেলেছিল। কিন্তু তা সত্ত্বেও এর test error (7.93%) ResNet-110 (6.43%)-এর চেয়ে বেশি হয়ে যায়।

এখানে সমস্যাটি আর degradation problem নয়, বরং Overfitting। CIFAR-10 ডেটাসেট (মাত্র ৫০,০০০ প্রশিক্ষণ ছবি) এত বিশাল প্যারামিটার সংখ্যার (১৯.৪ মিলিয়ন) একটি নেটওয়ার্ককে যথাযথভাবে সাধারণীকরণ (generalize) করতে সহায়তা করার জন্য যথেষ্ট বড় নয়। ফলে নেটওয়ার্কটি প্রশিক্ষণ ডেটার খুঁটিনাটি (এমনকি নয়েজ পর্যন্ত) মুখস্থ করে ফেলে, কিন্তু নতুন (unseen) ডেটাতে ভালোভাবে সাধারণীকরণ করতে ব্যর্থ হয়। এটি স্পষ্ট করে যে Residual Learning degradation problem সমাধান করলেও, এটি সীমাহীন গভীরতাকে যেকোনো ডেটাসেটে স্বয়ংক্রিয়ভাবে ভালো করে তোলে না — উপযুক্ত গভীরতা নির্বাচন এখনও ডেটাসেটের আকারের উপর নির্ভরশীল।



চিত্র ৪: CIFAR-10-এ বিভিন্ন গভীরতার ResNet — training/test error বনাম গভীরতা (ResNet-110 ও ResNet-1202)

অংশ ২১: Layer Response বিশ্লেষণ (Figure 7)

Original Paper-এর Figure 7-এ লেখকরা প্রতিটি স্তরের output response (মূলত প্রতিটি 3×3 convolution স্তরের output-এর standard deviation) পরিমাপ করে plain network এবং ResNet-এর মধ্যে তুলনা করেছেন।

মূল পর্যবেক্ষণ:

✍️ Residual Network-এর স্তরগুলোর response সাধারণত সংশ্লিষ্ট (corresponding) plain network-এর স্তরগুলোর তুলনায় ছোট (smaller) হয় (Original Paper, Figure 7)।

এর স্বজ্ঞাত (intuitive) ব্যাখ্যা:

- Plain Network: প্রতিটি স্তরকে সম্পূর্ণ নতুন একটি রূপান্তর (transformation) শিখতে হয় — ফলে তার response তুলনামূলক বড় হতে পারে।
- Residual Network: প্রতিটি residual ব্লককে কেবল input-এর সাপেক্ষে একটি ছোট সংশোধন (correction), $F(x)$, শিখলেই চলে — ফলে $F(x)$ -এর মান (response) সাধারণত ছোট হয়, বিশেষত যখন optimal mapping identity-এর কাছাকাছি হয়।

এই পর্যবেক্ষণটি Residual Learning-এর মূল প্রকল্পকে (hypothesis) সরাসরি সমর্থন করে — অর্থাৎ ResNet-এর প্রতিটি স্তর বাস্তবেই ছোট ছোট সংশোধনমূলক (residual) ফাংশন শিখছে, সম্পূর্ণ নতুন রূপান্তর নয়।

অংশ ২২: Object Detection-এ ResNet-এর ব্যবহার

ResNet কেবল classification-এই সীমাবদ্ধ থাকেনি — এটি Object Detection-এর জন্য একটি backbone (feature extractor) হিসেবেও ব্যবহৃত হয়েছে। ব্যবহৃত মূল detection framework ছিল Faster R-CNN:

Image → ResNet Feature Extractor → Faster R-CNN → Region Proposals → Classification + Bounding Box Regression

এই পাইপলাইনের গুরুত্বপূর্ণ উপাদানসমূহ:

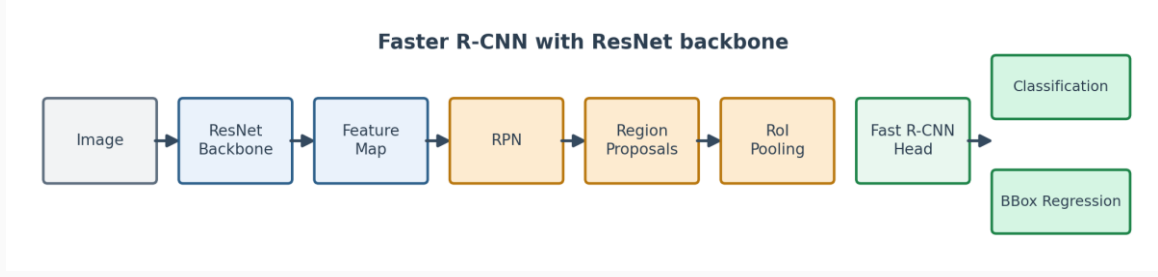
- RPN (Region Proposal Network): ছবির কোন কোন অঞ্চলে বস্তু থাকতে পারে তার সম্ভাব্য প্রস্তাবনা (region proposal) তৈরি করে।
- Region Proposal: RPN-এর দেওয়া প্রতিটি সম্ভাব্য bounding box, যেখানে হয়তো একটি বস্তু আছে।
- RoI Pooling (Region of Interest Pooling): বিভিন্ন আকারের region proposal-কে একটি নির্দিষ্ট, স্থির আকারের feature map-এ রূপান্তর করে, যাতে পরবর্তী স্তরে একইভাবে প্রক্রিয়া করা যায়।
- Fast R-CNN: RoI-ভিত্তিক feature ব্যবহার করে প্রতিটি proposal-এর জন্য শ্রেণি (class) ও bounding box নির্ধারণ করে।
- Bounding Box Regression: বস্তুর চারপাশের আয়তক্ষেত্র (rectangle)-এর সঠিক অবস্থান ও আকার নির্ণয়ের গাণিতিক প্রক্রিয়া।
- mAP (mean Average Precision): object detection-এর নির্ভুলতা পরিমাপের একটি প্রমিত (standard) মেট্রিক, যা বিভিন্ন শ্রেণি ও IoU (Intersection over Union) থ্রেশহোল্ডে গড় নির্ভুলতা পরিমাপ করে।

২২.১ VGG-16 বনাম ResNet-101 তুলনা

Original Paper-এর Section 4.3 অনুযায়ী, একই Faster R-CNN detection পদ্ধতিতে backbone হিসেবে শুধু VGG-16-কে ResNet-101 দিয়ে প্রতিস্থাপন করার ফলে COCO ডেটাসেটের প্রমিত মেট্রিক mAP@[.5, .95]-এ ৬.০ শতাংশ-বিন্দু (percentage point) উন্নতি দেখা যায়, যা একটি ২৮% আপেক্ষিক (relative) উন্নতির সমতুল্য।

☞ যেহেতু detection pipeline-এর বাকি সব উপাদান অপরিবর্তিত রাখা হয়েছিল, এই উন্নতির সম্পূর্ণ কৃতিত্ব দেওয়া যায় ResNet-এর শক্তিশালী feature representation-কে — এটি প্রমাণ করে যে Residual Learning-এর মাধ্যমে শেখা representation শুধু classification নয়, বরং বিভিন্ন downstream computer vision কাজেও (task) হস্তান্তরযোগ্য (transferable) ও কার্যকর।

এই ফলাফলের ভিত্তিতে ResNet-ভিত্তিক পদ্ধতি ILSVRC ও COCO ২০১৫ প্রতিযোগিতার একাধিক বিভাগে (ImageNet detection, ImageNet localization, COCO detection, COCO segmentation) প্রথম স্থান অর্জন করে।



চিত্র 9: Faster R-CNN + ResNet backbone দিয়ে object detection pipeline

অংশ ২৩: Object Localization

ImageNet Localization পরীক্ষায় ResNet-101 ব্যবহার করে বস্তুর অবস্থান নির্ণয়ের সক্ষমতা যাচাই করা হয়। প্রথমেই Classification ও Localization-এর মধ্যে পার্থক্যটি স্পষ্ট করা প্রয়োজন:

	মূল প্রশ্ন
Classification (শ্রেণিবিন্যাস)	“ছবিতে কোন বস্তুটি আছে?”
Localization (অবস্থান নির্ণয়)	“বস্তুটি ছবির ঠিক কোথায় আছে?”

Localization-এ শুধু বস্তুর শ্রেণি নয়, বরং তার চারপাশের সঠিক bounding box-ও পূর্বাভাস দিতে হয়। এই কাজে ব্যবহৃত মূল উপাদানসমূহ:

- RPN: সম্ভাব্য বস্তুর অবস্থানের প্রস্তাবনা তৈরি করে
- Bounding Boxes: প্রতিটি প্রস্তাবিত অবস্থানের আয়তক্ষেত্রাকার সীমানা
- Per-class Regression: প্রতিটি শ্রেণির জন্য আলাদাভাবে bounding box-এর সূক্ষ্ম সমন্বয় (regression) করা হয়, যাতে ভিন্ন ভিন্ন বস্তুর আকৃতি অনুযায়ী নির্ভুলতা বাড়ে
- ResNet-101: এই পরীক্ষায় feature extractor হিসেবে ব্যবহৃত মূল backbone
- R-CNN-ধর্মী কাঠামো: region-ভিত্তিক প্রস্তাবনা ও classification-এর সমন্বয়ে গঠিত সম্পূর্ণ পদ্ধতি

ResNet-101-ভিত্তিক এই পদ্ধতি ব্যবহার করে দলটি ImageNet Localization প্রতিযোগিতায় শীর্ষ স্থান অর্জন করে, যা আরও একবার প্রমাণ করে যে গভীর Residual Network থেকে শেখা feature representation classification ছাড়াও অন্যান্য জটিল ভিশন কাজে অত্যন্ত কার্যকর।

📄 Original Paper, Appendix C; Tables 13 and 14

অংশ ২৪: গুরুত্বপূর্ণ পরিভাষা (Glossary)

পরিভাষা	সংক্ষিপ্ত সংজ্ঞা
CNN	Convolutional Neural Network — ছবি-জাতীয় ডেটা থেকে বৈশিষ্ট্য শেখার জন্য বিশেষভাবে তৈরি একটি নিউরাল নেটওয়ার্ক স্থাপত্য
Feature Map	Convolution অপারেশনের ফলে তৈরি হওয়া নতুন গ্রিড, যা নির্দিষ্ট বৈশিষ্ট্যের উপস্থিতি নির্দেশ করে
Channel	একটি feature map-এর একটি স্তর/স্লাইস, যা একটি নির্দিষ্ট ধরনের প্যাটার্ন ধারণ করে
Kernel	Convolution-এ ব্যবহৃত ছোট সংখ্যার গ্রিড, যা ইনপুটের উপর দিয়ে সরে বৈশিষ্ট্য বের করে
Filter	একাধিক kernel-এর সমষ্টি, যা একটি সম্পূর্ণ output channel তৈরি করে
Stride	Kernel প্রতিবার কত pixel সরবে তার পরিমাণ
Padding	Input-এর চারপাশে যোগ করা অতিরিক্ত pixel, যা output-এর আকার নিয়ন্ত্রণ করে
ReLU	Rectified Linear Unit; $\sigma(x) = \max(0, x)$ — একটি জনপ্রিয় non-linear activation function
Batch Normalization	মিনি-ব্যাচ ভিত্তিতে স্তরের output normalize করার কৌশল, যা প্রশিক্ষণ স্থিতিশীল করে
Residual	কান্ডাক্ত output এবং input-এর মধ্যকার পার্থক্য $(H(x) - x)$
Residual Function	$F(x)$ — নেটওয়ার্ক যা শিখতে চেষ্টা করে, যা কান্ডাক্ত ম্যাপিং $H(x)$ -এর সাথে input x -এর পার্থক্য
Identity Mapping	$x \rightarrow x$ — input-কে অবিকৃতভাবে output-এ পাঠানো
Shortcut Connection	এক বা একাধিক স্তর অতিক্রম করে সরাসরি সংযোগকারী পথ
Skip Connection	Shortcut Connection-এর অপর নাম
Projection Shortcut	1×1 convolution ব্যবহার করে dimension মেলানোর জন্য তৈরি শর্টকাট (W_{sx})
1×1 Convolution	1×1 আকারের kernel-বিশিষ্ট convolution, যা channel dimension রূপান্তর করে
Bottleneck	$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ কাঠামোর একটি সংকীর্ণ-প্রশস্ত-সংকীর্ণ residual block
Global Average Pooling	প্রতিটি feature map-এর সম্পূর্ণ স্থানিক অংশের গড় নিয়ে একটিমাত্র মানে পরিণত করা
Plain Network	কোনো shortcut connection ছাড়া স্তরসমূহের সরল স্তূপ
ResNet	Residual Network — শর্টকাট সংযোগযুক্ত অত্যন্ত গভীর CNN স্থাপত্য
Top-1 Error	সর্বোচ্চ সম্ভাবনার পূর্বাভাস সঠিক না হলে ভুল গণনা করা হয়
Top-5 Error	প্রথম ৫টি পূর্বাভাসের মধ্যে সঠিক উত্তর না থাকলে ভুল গণনা করা হয়
FLOPs	Floating Point Operations — একটি মডেলের গণনার পরিমাণ পরিমাপের একক
ImageNet	১০০০ শ্রেণির ১০ লক্ষাধিক ছবির একটি বৃহৎ classification ডেটাসেট

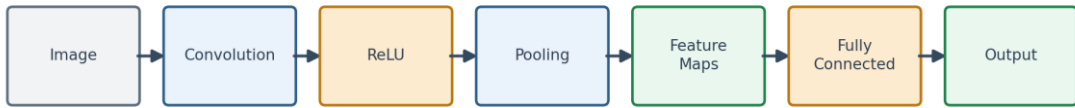
পরিভাষা	সংক্ষিপ্ত সংজ্ঞা
CIFAR-10	১০ শ্রেণির ৩২×৩২ আকারের ছোট ছবির একটি ডেটাসেট
Faster R-CNN	RPN ব্যবহার করে দ্রুত region proposal তৈরি করা একটি object detection framework
RPN	Region Proposal Network — সম্ভাব্য বস্তুর অবস্থানের প্রস্তাবনা তৈরি করে
RoI Pooling	Region of Interest Pooling — বিভিন্ন আকারের region-কে স্থির আকারের feature-এ রূপান্তর করে
mAP	mean Average Precision — object detection-এর নির্ভুলতা পরিমাপের প্রমিত মেট্রিক

অংশ ২৫: ভিজ্যুয়াল ডায়াগ্রাম (Visual Diagrams)

এই অংশে ResNet ও CNN সংক্রান্ত মূল ধারণাগুলোর জন্য প্রয়োজনীয় ডায়াগ্রামের একটি তালিকা দেওয়া হলো। এই নোটের পূর্ববর্তী অংশগুলোতেও প্রাসঙ্গিক স্থানে একই ধরনের ডায়াগ্রাম-স্থান (চিত্র ১ থেকে ৯) চিহ্নিত করা হয়েছে। নিচে অবশিষ্ট তিনটি মৌলিক ডায়াগ্রামের স্থান, এবং সম্পূর্ণ চিত্র-তালিকার সারসংক্ষেপ দেওয়া হলো।

২৫.১ Diagram: মৌলিক CNN পাইপলাইন

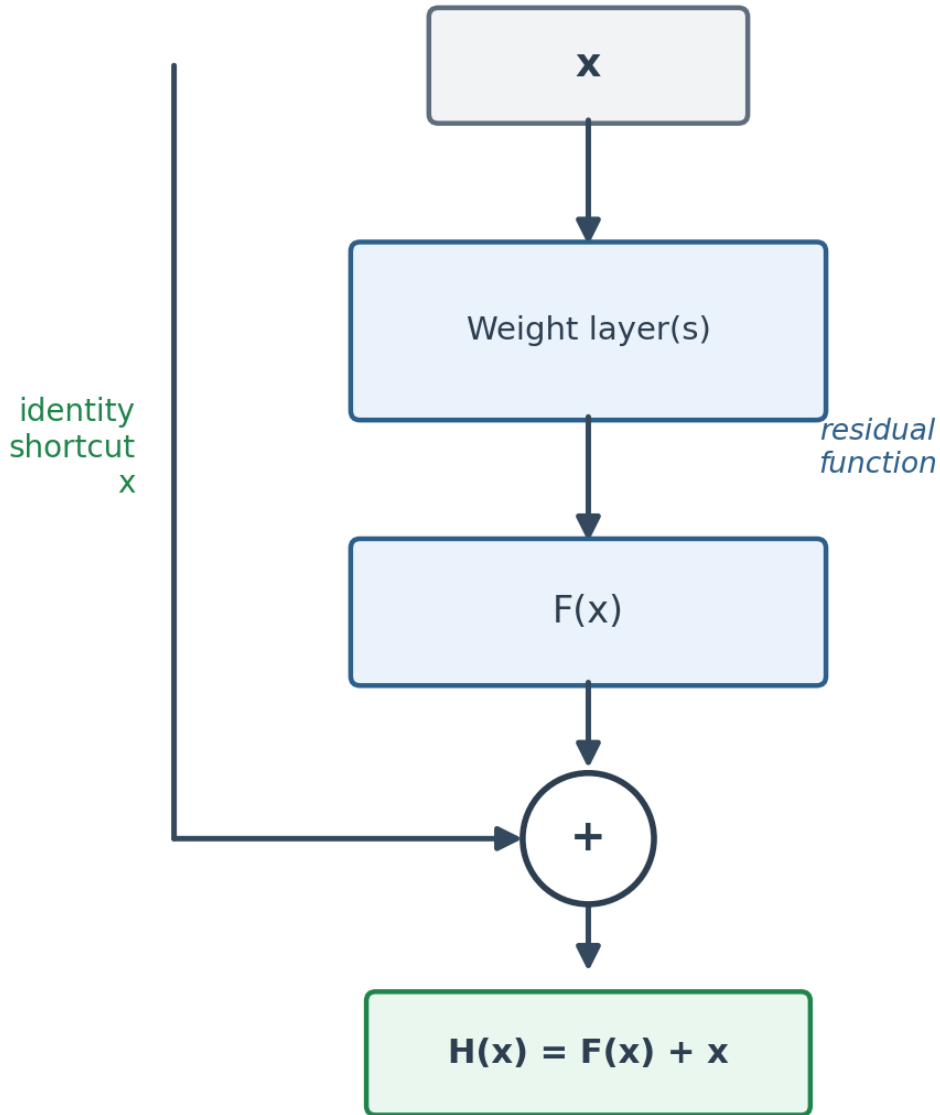
Image → Convolution → ReLU → Pooling → Feature Maps → Fully Connected → Output



চিত্র 10: মৌলিক CNN পাইপলাইন — ছবি থেকে চূড়ান্ত আউটপুট পর্যন্ত ধাপে ধাপে প্রবাহ

২৫.২ Diagram: $F(x) + x$ ধারণাগত চিত্র

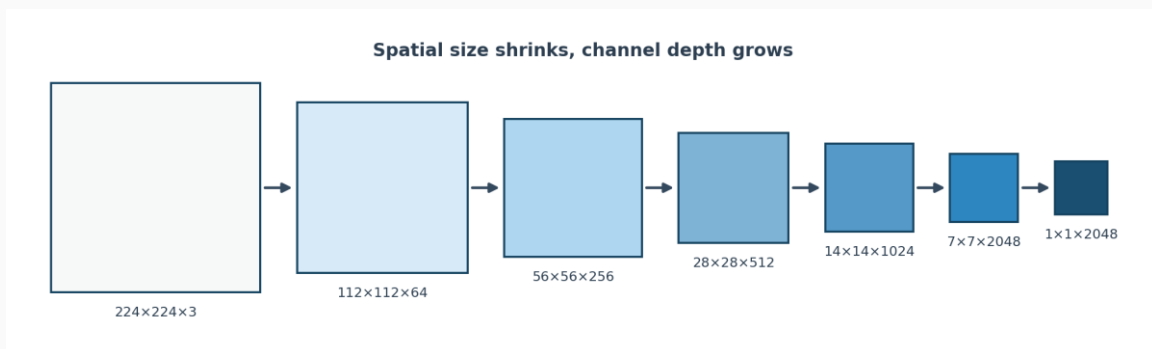
Residual Learning-এর মূল গাণিতিক ধারণা x থেকে $F(x)$ এবং শেষে যোগফল $F(x)+x$ পর্যন্ত একটি প্রবাহচিত্রে দেখানো যায়।



চিত্র 11: $x \rightarrow F(x) \rightarrow F(x)+x$ — Residual Learning-এর ধারণাগত চিত্র

২৫.৩ Diagram: স্থানিক আকার ও Channel-এর সহ-পরিবর্তন

$224 \times 224 \times 3 \rightarrow 112 \times 112 \times 64 \rightarrow 56 \times 56 \times 256 \rightarrow 28 \times 28 \times 512 \rightarrow 14 \times 14 \times 1024 \rightarrow 7 \times 7 \times 2048 \rightarrow 1 \times 1 \times 2048$



চিত্র 12: ResNet-50-এ স্থানিক আকার হ্রাস ও channel বৃদ্ধির সহ-পরিবর্তন

২৫.৪ সম্পূর্ণ চিত্র-তালিকা (সারসংক্ষেপ)

চিত্র নম্বর	বিষয়	যেখানে আছে
১	Plain vs Residual — Degradation Problem	অংশ ৪
২	Residual Block (main path + shortcut)	অংশ ৬
৩	Dimension Mismatch ও Projection Shortcut	অংশ ৯
৪	1×1 Convolution — channel রূপান্তর	অংশ ১০
৫	Bottleneck Block গঠন	অংশ ১১
৬	ResNet-50 সম্পূর্ণ স্থাপত্য	অংশ ১৪
৭	Plain vs ResNet training/validation curve	অংশ ১৭
৮	CIFAR-10 গভীরতা বনাম error curve	অংশ ২০
৯	Object Detection Pipeline (Faster R-CNN)	অংশ ২২
১০	মৌলিক CNN পাইপলাইন	অংশ ২৫.১
১১	$F(x)+x$ ধারণাগত চিত্র	অংশ ২৫.২
১২	স্থানিক আকার ও Channel সহ-পরিবর্তন	অংশ ২৫.৩

অংশ ২৬: গাণিতিক সারাংশ (Mathematical Summary)

এই নোটে ব্যবহৃত সবচেয়ে গুরুত্বপূর্ণ সমীকরণগুলো এক জায়গায় সংকলিত করা হলো, প্রতিটির সংক্ষিপ্ত ব্যাখ্যাসহ।

$$F(x) = H(x) - x$$

নেটওয়ার্ক যে residual function শেখে, তা হলো কাঙ্ক্ষিত ম্যাপিং $H(x)$ এবং input x -এর মধ্যকার পার্থক্য।

$$H(x) = F(x) + x$$

residual function এবং input যোগ করে মূল কাঙ্ক্ষিত ম্যাপিং $H(x)$ পুনর্গঠিত হয় — এটিই Residual Block-এর output।

$$y = F(x, \{W_i\}) + x$$

সাধারণ আকারে Residual Block-এর সমীকরণ, যেখানে $\{W_i\}$ হলো ঐ ব্লকের ভেতরের সব convolution স্তরের weight-সমূহ।

$$y = F(x, \{W_i\}) + W_s x$$

Dimension না মিললে ব্যবহৃত projection shortcut-সহ সমীকরণ, যেখানে W_s একটি 1×1 convolution projection।

$$F(x) = W_2 \sigma(W_1 x)$$

একটি দুই-স্তরবিশিষ্ট residual function-এর উদাহরণ — প্রথমে W_1 প্রয়োগ, তারপর ReLU, তারপর W_2 প্রয়োগ।

$$\sigma(x) = \max(0, x)$$

ReLU activation function — ঋণাত্মক মানকে শূন্য করে, ধনাত্মক মান অপরিবর্তিত রাখে।

$$\text{Output size} = \lfloor (N + 2P - K) / S \rfloor + 1$$

Convolution বা Pooling-এর পর output feature map-এর height/width নির্ণয়ের সূত্র, যেখানে N =input আকার, P =padding, K =kernel আকার, S =stride।

✍ এই নোটের সব সমীকরণ যথাযথ mathematical notation-সহ (Unicode subscript/superscript ও প্রতীক ব্যবহার করে) উপস্থাপন করা হয়েছে যাতে তা কোনোভাবেই raw LaTeX কোড হিসেবে না দেখায় এবং একটি প্রমিত পাঠ্যপুস্তকের সমীকরণের মতো দেখতে হয়।

অংশ ২৭: স্বজ্ঞাত সংখ্যাগত উদাহরণ (Intuitive Examples)

উদাহরণ ১: Residual Learning

$$x = 10, H(x) = 12 \Rightarrow F(x) = H(x) - x = 2$$

নেটওয়ার্ককে সম্পূর্ণ রূপান্তর ($10 \rightarrow 12$) শেখার বদলে কেবল +2 সংশোধন শিখতে হয়।

উদাহরণ ২: Element-wise Addition

$$F(x) = [1, 2, 3], x = [10, 20, 30] \Rightarrow F(x) + x = [11, 22, 33]$$

প্রতিটি সংশ্লিষ্ট (corresponding) অবস্থানের মান পৃথকভাবে যোগ হয়।

উদাহরণ ৩: 1×1 Convolution দিয়ে Channel হ্রাস

$$56 \times 56 \times 256 \rightarrow 1 \times 1 \text{ Conv} \rightarrow 56 \times 56 \times 64$$

স্থানিক আকার (height, width) অপরিবর্তিত থেকে শুধু channel সংখ্যা কমেছে।

উদাহরণ ৪: Bottleneck-এর Channel প্রবাহ

$$256 \rightarrow 64 \rightarrow 64 \rightarrow 256$$

প্রথমে সংকোচন ($256 \rightarrow 64$), মাঝে 3×3 convolution ($64 \rightarrow 64$), শেষে প্রসারণ ($64 \rightarrow 256$)।

উদাহরণ ৫: Spatial Downsampling

$$56 \times 56 \rightarrow \text{Stride } 2 \rightarrow 28 \times 28$$

Stride 2 ব্যবহার করলে height ও width উভয়ই অর্ধেক হয়ে যায়।

অংশ ২৮: মূল গবেষণাপত্রের Figure/Table নির্দেশিকা

এই নোট জুড়ে যেখানেই কোনো ধারণা সরাসরি মূল গবেষণাপত্র থেকে নেওয়া হয়েছে, সেখানে তার উৎস (Section, Figure বা Table নম্বর) উল্লেখ করা হয়েছে। সম্পূর্ণ তালিকাটি একত্রে নিচে দেওয়া হলো, দ্রুত খুঁজে পাওয়ার সুবিধার্থে:

উৎস	বিষয়বস্তু
Section 1; Figure 1	Degradation Problem — গভীর plain network-এর উচ্চ training error
Section 3.1	Residual Learning-এর মূল ধারণা ও যুক্তি
Section 3.2; Figure 2	Residual Block-এর গঠন
Section 3.4	প্রশিক্ষণের বিস্তারিত (implementation) সেটআপ
Section 4.1; Table 1	বিভিন্ন গভীরতার ResNet স্থাপত্য
Section 4.1; Figure 4; Table 2	Plain বনাম ResNet — ImageNet ফলাফল
Section 4.1; Table 3	Identity বনাম Projection Shortcut (Option A/B/C)
Table 3, 4	গভীরতর ResNet (৫০/১০১/১৫২)-এর ফলাফল
Figure 5	Bottleneck Block-এর গঠন
Section 4.2; Table 6; Figure 6	CIFAR-10 পরীক্ষা ও অত্যন্ত গভীর নেটওয়ার্কের আচরণ
Figure 7	Layer Response বিশ্লেষণ
Section 4.3; Appendix A; Tables 7–11	Object Detection (PASCAL VOC ও MS COCO)
Appendix C; Tables 13, 14	ImageNet Localization

এই নোটে উপস্থাপিত সব পরীক্ষামূলক ফলাফল (সংখ্যা, শতাংশ) মূল ২০১৫ সালের গবেষণাপত্র থেকে সরাসরি নেওয়া ঐতিহাসিক (historical) ফলাফল। কোনো সংখ্যা অনুমান করে বা নতুন করে তৈরি করা হয়নি।

অংশ ২৯: CNN ধারণা → ResNet-এ প্রয়োগ

এই সারণিটি দেখায় যে ResNet মোটেও CNN থেকে সম্পূর্ণ ভিন্ন কোনো ধারণা নয় — বরং এটি পরিচিত CNN মৌলিক উপাদানগুলোর উপর ভিত্তি করে তৈরি একটি স্থাপত্যগত (architectural) উন্নতি।

CNN ধারণা	ResNet-এ যেভাবে ব্যবহৃত হয়েছে
Convolution	প্রতিটি স্তরে feature নিষ্কাশন (extraction)
ReLU	প্রতিটি ব্লকে non-linearity যোগ করা
Batch Normalization	প্রশিক্ষণ স্থিতিশীল রাখা
Stride	স্থানিক downsampling (আকার হ্রাস)
1×1 Conv	Channel dimension রূপান্তর ও bottleneck তৈরি
3×3 Conv	মূল স্থানিক (spatial) বৈশিষ্ট্য নিষ্কাশন

CNN ধারণা	ResNet-এ যেভাবে ব্যবহৃত হয়েছে
Global Average Pooling	চূড়ান্ত feature সংক্ষিপ্তকরণ (aggregation)
Shortcut / Skip Connection	Residual Learning সক্ষম করা — ResNet-এর মূল অবদান

এই তুলনা থেকে স্পষ্ট হয় — ResNet-এর মূল উদ্ভাবন (innovation) হলো শুধু একটি অতিরিক্ত উপাদান, shortcut connection, যা আগে থেকে পরিচিত CNN উপাদানগুলোর (convolution, ReLU, BN, pooling) সাথে যুক্ত করা হয়েছে। এই সরল অথচ শক্তিশালী সংযোজনটিই অত্যন্ত গভীর নেটওয়ার্ককে কার্যকরভাবে প্রশিক্ষণযোগ্য করে তুলেছে।

অংশ ৩০: সম্পূর্ণ ResNet ধারণা — একটি প্রবাহে

Input Image → CNN Feature Extraction → Residual Block ($F(x)+x$) → আরও Residual Blocks → (Spatial ↓, Channel ↑) → Global Average Pooling → Feature Vector → Fully Connected → Softmax → Class Prediction

সংক্ষেপে বলা যায় — একটি ছবি প্রথমে প্রাথমিক convolution স্তর দিয়ে সাধারণ বৈশিষ্ট্য শনাক্তকরণের মধ্য দিয়ে যায়; এরপর একের পর এক Residual Block-এর মধ্য দিয়ে অতিক্রম করে, যেখানে প্রতিটি ব্লক তার input-এর সাপেক্ষে একটি ছোট সংশোধনমূলক (residual) পরিবর্তন যোগ করে, এবং shortcut connection-এর মাধ্যমে মূল তথ্য অক্ষত থেকে সামনে প্রবাহিত হতে থাকে। নেটওয়ার্ক যত গভীরে যায়, স্থানিক আকার (height, width) ততই ছোট হতে থাকে এবং channel সংখ্যা ততই বাড়তে থাকে — অর্থাৎ কম বিস্তারিত কিন্তু বেশি বিমূর্ত তথ্যে রূপান্তরিত হতে থাকে। সবশেষে Global Average Pooling সমস্ত স্থানিক তথ্যকে একটিমাত্র feature vector-এ সংক্ষিপ্ত করে, যা একটি ছোট Fully Connected স্তর ও Softmax-এর মাধ্যমে চূড়ান্ত শ্রেণি পূর্বাভাসে রূপান্তরিত হয়। এই সম্পূর্ণ প্রক্রিয়াটিই একটি নেটওয়ার্ককে একই সাথে অত্যন্ত গভীর ও কার্যকরভাবে প্রশিক্ষণযোগ্য হতে সাহায্য করে।

অংশ ৩১: গবেষণাপত্রের চূড়ান্ত সারাংশ

ধাপ	বিবরণ
সমস্যা (Problem)	অত্যন্ত গভীর plain network অপ্টিমাইজ করা কঠিন হয়ে পড়ে
পর্যবেক্ষণ (Observation)	গভীরতা বাড়াতে training error বাড়তে পারে (degradation problem)
ধারণা (Idea)	সরাসরি $H(x)$ না শিখে residual $F(x)$ শেখা
Residual	$F(x) = H(x) - x$
Shortcut	x সরাসরি পরবর্তী ধাপে পাঠানো
Addition	$F(x) + x$
ফলাফল (Result)	নেটওয়ার্ক residual সংশোধন অনেক সহজে শিখতে পারে
স্থাপত্য (Architecture)	অনেকগুলো Residual Block স্তপ করে অত্যন্ত গভীর নেটওয়ার্ক তৈরি

ধাপ	বিবরণ
Bottleneck	$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ — গণনাগতভাবে সাশ্রয়ী গভীর ব্লক
ফলাফল (Result)	ResNet-50/101/152 ব্যবহারযোগ্য গণনার মধ্যেই অনেক গভীরতা অর্জন করে
প্রয়োগ (Applications)	Classification, Object Detection, Localization, Segmentation

অংশ ৩২: চূড়ান্ত এক-পৃষ্ঠার Revision Sheet

মূল সমস্যা (Core Problem)

অত্যন্ত গভীর plain network-এ Degradation।

মূল ধারণা (Core Idea)

Residual Learning।

মূল সমীকরণ (Core Equation)

$$H(x) = F(x) + x$$

Shortcut

Identity Mapping।

Basic Block

$$3 \times 3 \rightarrow 3 \times 3$$

Bottleneck Block

$$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$$

1×1 Conv

দক্ষভাবে channel dimension পরিবর্তন করে।

GAP

$$7 \times 7 \times 2048 \rightarrow 1 \times 1 \times 2048$$

প্রধান স্থাপত্যসমূহ (Main Architectures)

- ResNet-18
- ResNet-34
- ResNet-50
- ResNet-101
- ResNet-152

প্রধান প্রমাণ (Main Evidence)

পরীক্ষায় সমান গভীরতার plain network-এর তুলনায় গভীর ResNet ভালোভাবে প্রশিক্ষিত হয়েছে।

মূল অবদান (Main Contribution)

Residual Learning উল্লেখযোগ্যভাবে গভীরতর CNN-কে অপ্টিমাইজ করা সহজ করে তুলেছে।

তথ্যসূত্র (References)

- [১] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” Microsoft Research, 2015 (arXiv:1512.03385). — এই নোটের মূল উৎস গবেষণাপত্র।
- [২] O. Russakovsky et al., “ImageNet Large Scale Visual Recognition Challenge,” IJCV, 2015. — ImageNet ডেটাসেট সংক্রান্ত তথ্যসূত্র।
- [৩] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” 2009. — CIFAR-10 ডেটাসেট সংক্রান্ত তথ্যসূত্র।
- [৪] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” NeurIPS, 2015. — Object Detection framework সংক্রান্ত তথ্যসূত্র।